

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 5330003

ANDHRA PRADESH

WEATHER ALERTS APP

By

KOLLU CHINNARI SAI BABU

(Regd. No. 24VV1A1232)

Under the supervision of

Mrs. MADHUMITA CHANDA

Mrs. GEETHA MADHURI YADAV

Department of Information Technology

Viziangaram – 533003



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 535003

ANDHRA PRADESH

Regd.No:24VV1A1232

This is to certify that this is a bonafide record of practical work done by KOLLU CHINNARI SAI BABU of III B.Tech I Semester Class in USER INTERFACE DESIGN USING FLUTTER Lab during the year 2026-27.

No.of Tasks Completed and Certified:

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

INDEX

S.NO	CONCEPT	PAGE NO.	Marks	Signature
1	Introduction	1		
2	Project Structure	2		
3	Installing Firebase Studio	3 – 4		
4	Installing Flutter SDK	5 – 8		
5	Installing Dart SDK	9		
6	Installing VS Code	10		
7	Create a New Flutter Project	11 – 12		
8	Dart Basic Programs	13 – 15		
9	Case Study	16		
10	Flutter Widgets	17 – 25		
11	Responsiveness UI Design	26 – 30		
12	Navigation Between Screens	31		
13	Stateful and Stateless Widgets	32 – 33		
14	Custom Widgets and Styling	34 – 37		
15	Forms and Validation	38 – 46		
16	Animations	47 – 52		
17	REST API Integration	53		
18	Testing and Debugging	54 – 55		
19	Flow Diagram of Streetlight Complaint Application	56 – 59		
20	Conclusion	60		

1 Introduction

Flutter is an open-source software development framework created by Google. It allows developers to build high-quality, natively compiled applications for mobile, web, and desktop from a single codebase.

Advantages of Flutter

- Cross-platform development with a single codebase.
- Hot reload for instant updates.
- Rich set of customizable widgets.
- High performance due to AOT compilation.
- Backed by Google and strong community support.

Dart Language

Dart is the programming language used by Flutter. It is object-oriented, class-based, and supports JIT and AOT compilation. Dart syntax is simple and similar to Java/C++.

Advantages of Dart

- Easy-to-learn syntax.
- Null safety to avoid runtime errors.
- Strong typing and async/await support.
- JIT for fast development, AOT for optimized production.

Key Features of Flutter

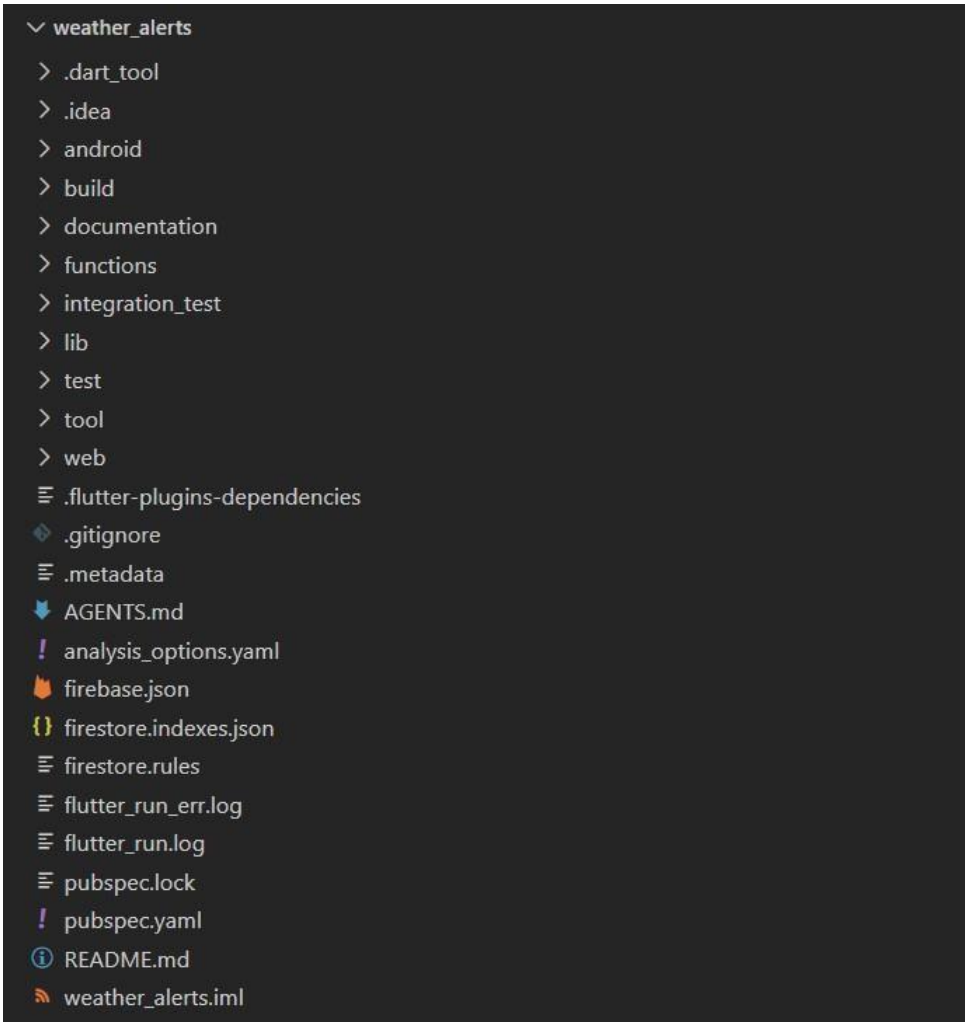
- **Cross-Platform Development:** One codebase can run on multiple platforms such as Android, iOS, Web, Windows, macOS, and Linux.
- **Hot Reload:** Instantly view code changes without restarting the app, improving developer productivity and speeding up testing.
- **Widget-Based Framework:** Everything in Flutter is a widget — layouts, buttons, text, and images, which makes UI creation simple and flexible.
- **High Performance:** Flutter apps are compiled to native ARM code, ensuring fast performance and smooth animations.
- **Custom UI Design:** Allows the creation of flexible and visually appealing interfaces using Material Design (for Android) or Cupertino (for iOS) widgets.

Flutter follows a declarative UI model. Instead of modifying individual interface elements imperatively, the developer describes the required UI for a given state, and Flutter rebuilds the affected part of the widget tree when that state changes. This model reduces the risk of inconsistent screens and makes complex interfaces easier to reason about, because the visible output is always derived from the current application state.

2 Project Structure

The Quiz Leaderboard App is a Flutter-based mobile application designed to create an interactive quiz experience for users. It allows them to register, log in, participate in quizzes, and track their scores in a global leaderboard. The app demonstrates real-world Flutter practices including navigation, widget composition, form validation, state management, and integration with backend APIs.

Below is the directory structure of the project:



A Flutter project separates application code from platform-specific configuration. The lib directory contains Dart source files, while android, ios, web, windows, macos, and linux contain the host project files required by each target platform. The test directory holds automated tests, and pubspec.yaml records dependencies, assets, and project metadata. This separation keeps business logic portable while allowing platform-specific settings to be adjusted independently.

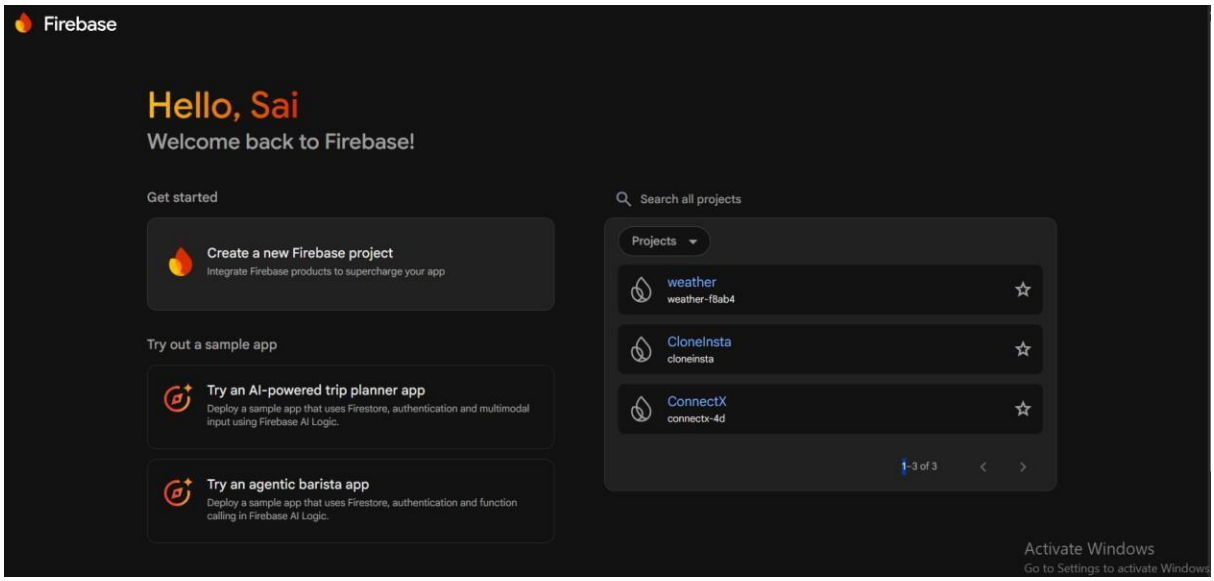
In the Weather Alerts application, the lib directory can be organised into screens, widgets, models, services, and utilities. Screens define complete pages, widgets define reusable interface components, models describe weather and alert data, and services handle authentication, API requests, and persistence. This organisation improves readability and makes it easier to locate and modify a feature.

3 Firebase Installation

Firebase is a platform developed by Google for creating mobile and web applications. It provides services like real-time databases, authentication, cloud storage, and more. The steps to install and configure Firebase for a Flutter project are as follows:

1. Create a Firebase Project:

- Go to the Firebase Console.
- Click on Add Project and follow the on-screen instructions.



Firebase Console dashboard.

2. Add Your App to Firebase:

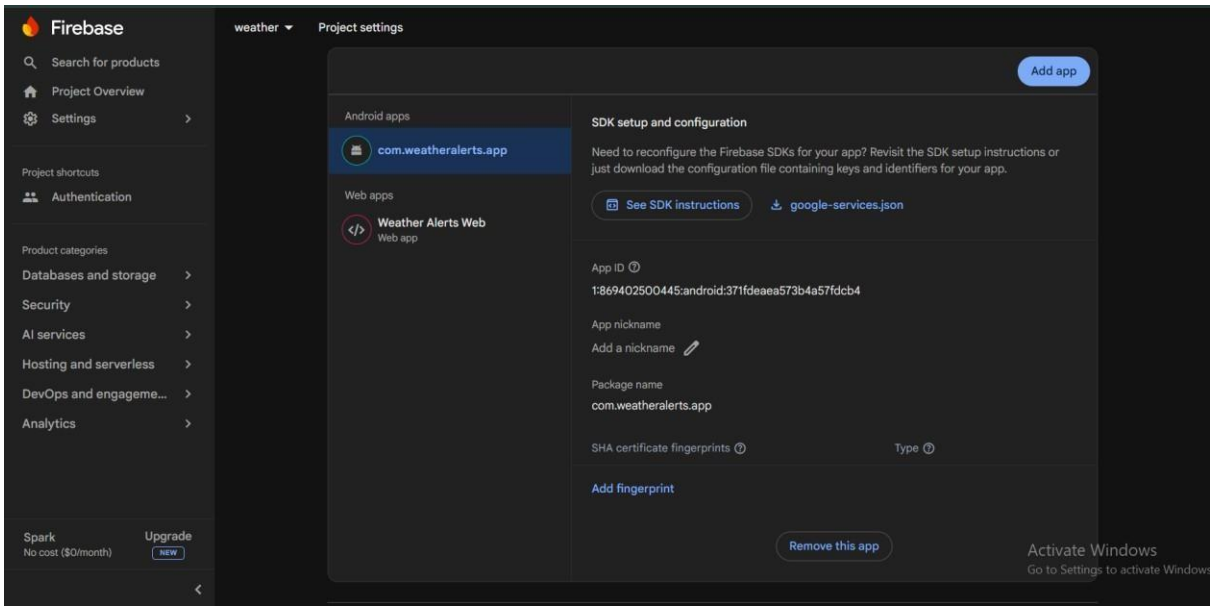
- Choose the platform (Android, iOS, or Web).
- For Android, provide the package name and optionally the app nickname and SHA-1 key.
- Download the google-services.json (for Android) or GoogleService-Info.plist (for iOS) and add it to your project.

3. Add Firebase SDK to Your Project:

- Open the pubspec.yaml file and add Firebase dependencies:

Firebase Authentication provides a managed identity service that handles account creation, sign-in, password reset, and session persistence. The client application sends credentials to Firebase, which verifies them and returns a user identity that can be used by the rest of the application. This removes the need to store passwords directly in the application and reduces the security risks associated with custom authentication code.

Cloud Firestore is a document-oriented database that stores data in collections and documents. It supports real-time listeners, offline persistence, and security rules that control access at the document level. In a weather application, Firestore can store user profiles, saved locations, alert preferences, and other settings that must remain associated with an authenticated user.



Adding a new app in Firebase.

```
dependencies:
  firebase_core: ^2.24.0
  firebase_auth: ^5.16.0
  cloud_firestore: ^5.16.0
```

- Run flutter pub get to install the packages.

4. Initialize Firebase in Your App:

- In main.dart, ensure Firebase is initialized before running the app:

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}
```

5. Verify Installation:

- Run the app on an emulator or device.
- Check the console for successful Firebase initialization.

The call to WidgetsFlutterBinding.ensureInitialized() prepares the Flutter engine before plugins are used. Firebase plugins rely on platform channels, so this step ensures that the binding exists when Firebase.initializeApp() communicates with the native Android or iOS layer. The asynchronous main function then waits for initialisation to complete before runApp() starts the widget tree.

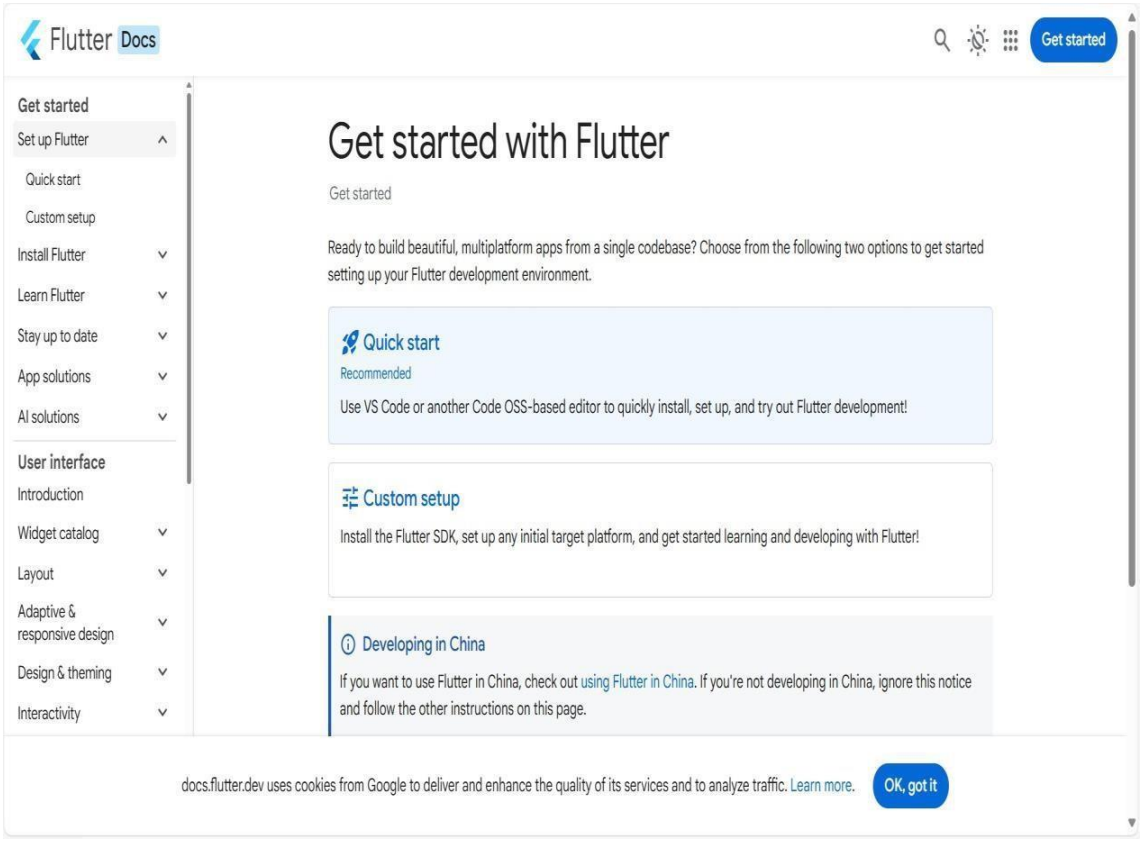
Firebase.initializeApp() reads the platform configuration and creates the default Firebase application instance. If the project uses multiple environments, named instances can be created, but a single default instance is sufficient for most academic projects. Initialisation should be performed once, near the start of the application, so that authentication and database services are ready when screens request them.

4 Installation of Flutter (with PATH Setup)

Download Flutter SDK

Step 1:

1. Visit: https://docs.flutter.dev/get-started/install.
2. Choose OS (Windows/Mac/Linux).
3. Download Flutter SDK (Stable Channel).
4. Extract it:
   - **Windows:** C:\src\flutter
   - **Linux/Mac:** $HOME/flutter



Insatllation of flutter

Add Flutter to PATH

Step 2:

Windows:

1. Go to Control Panel → System → Advanced system settings.
2. Environment Variables → Edit Path.

The Flutter SDK contains the command-line tools, framework packages, and platform templates needed to create and run applications. Adding the bin directory to the system PATH allows commands such as flutter, dart, and flutter doctor to be executed from any terminal location. Without the PATH entry, the operating system cannot locate the SDK executables.

3. Add: C:\src\flutter\bin.

Linux/Mac:

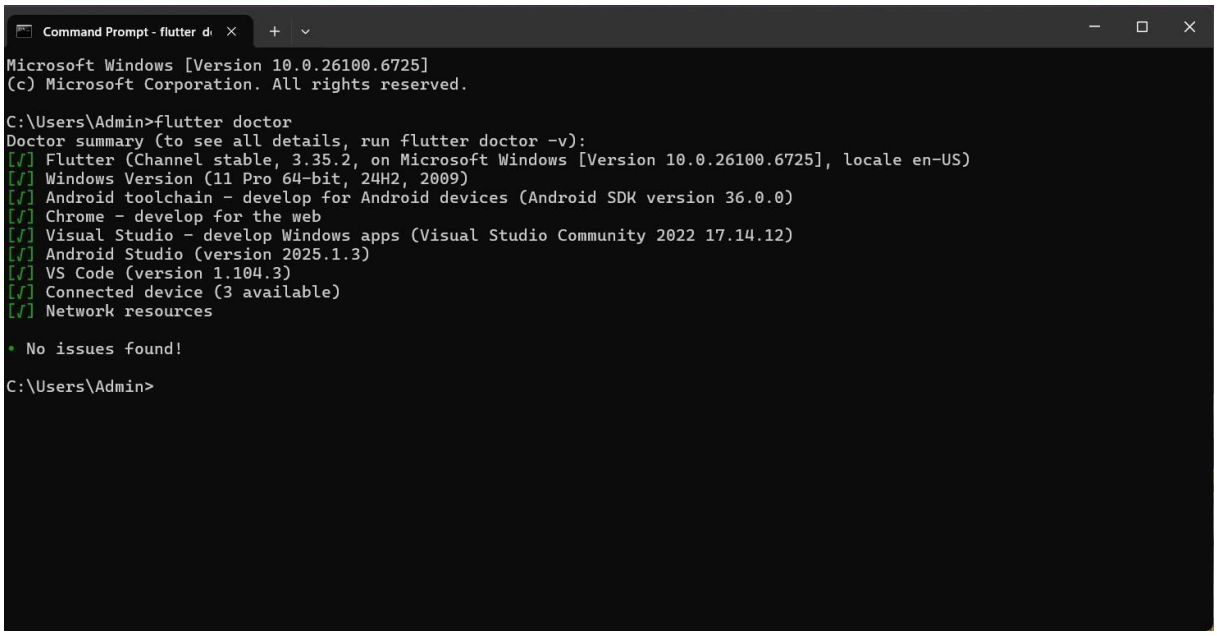
```
export PATH="$PATH:$HOME/flutter/bin"
```

Add this line to .bashrc or .zshrc.

Verify Installation

Step 3:

```
flutter doctor
```



flutter doctor Command Showing Environment Setup

The flutter doctor command performs a diagnostic check of the development environment. It verifies the Flutter SDK, the Dart SDK, the Android toolchain, Android Studio, connected devices, and the status of required licenses. Each reported issue includes a suggested command or configuration step, which makes the tool useful for both initial setup and troubleshooting.

A healthy environment usually reports that Flutter is installed, the Android toolchain is configured, and at least one device or emulator is available. Missing Android licenses can be accepted with flutter doctor --android-licenses, while an undetected device may require USB debugging, driver installation, or a restart of the emulator.

Consistent environment checks reduce unexplained build failures. Before running the Weather Alerts application, the developer should confirm that the Flutter version, Android SDK, and connected device are compatible. This practice saves time because configuration problems are identified before application code is tested.

Installation of Android Studio

Step 1: Download and Install

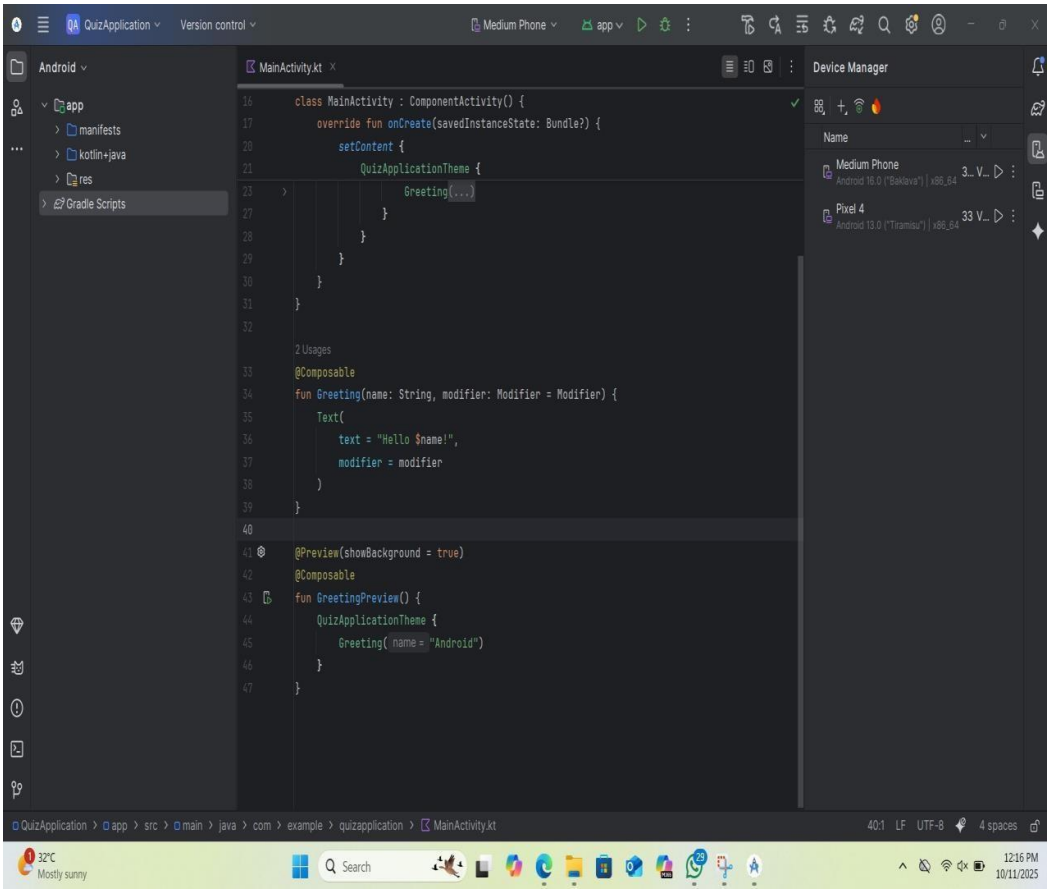
1. Go to: https://developer.android.com/studio.
2. Download Android Studio.
3. Install with default settings.

Step 2: Install Plugins

1. Open Android Studio.
2. Go to File → Settings → Plugins.
3. Install Flutter (Dart installs automatically).

Step 3: Configure SDK Path

1. Open SDK Manager.
2. Verify Android SDK location.
3. Install required SDK platforms & tools.



Android Studio Interface Showing Project Workspace

Android Studio is the official integrated development environment for Android development. It includes the Android SDK Manager, the AVD Manager, Gradle build support, a layout inspector, and a powerful debugger. Even when Flutter code is written in another editor, Android Studio remains useful for managing SDK platforms, emulators, and native Android configuration.

Emulator Setup and Running

1. Open Android Studio.
2. Go to Tools → AVD Manager.
3. Click Create Virtual Device.
4. Choose a device model (e.g., Pixel 6).
5. Select a system image (Android 13 or latest).
6. Finish and launch emulator.

An Android Emulator is a virtual Android device that runs on the development computer. It is created from an Android Virtual Device (AVD) definition, which specifies the device profile, system image, screen density, memory, storage, and hardware features. The emulator allows applications to be tested without a physical phone and makes it possible to simulate different screen sizes and Android versions.

A system image contains the Android operating system and the hardware abstraction layer used by the virtual device. Images are available for different API levels and CPU architectures. On Windows, hardware acceleration is typically provided by Windows Hypervisor Platform or a compatible hypervisor; without acceleration, the emulator may start slowly or fail to run.

The AVD configuration can be adjusted to match the target audience. Developers can choose a phone, tablet, foldable, or TV profile, set the amount of RAM and internal storage, configure the camera and network behaviour, and select the graphics rendering mode. These options are useful for testing responsive layouts and for reproducing issues that depend on device capabilities.

When testing a Flutter application, the emulator supports hot reload, debugging, widget inspection, and performance profiling. It is particularly convenient for checking navigation flows and form validation because the application can be restarted quickly. However, the emulator does not fully represent the performance or sensor behaviour of a physical device, so final testing should also include real hardware.

Good emulator practice includes using a cold boot when the virtual device behaves unexpectedly, keeping system images updated, and avoiding excessive resource allocation that can slow the host machine. Snapshots can reduce startup time, but they should be refreshed after major SDK or application changes to avoid inconsistent states.

Emulator performance depends on the host machine, the selected system image, and the graphics configuration. When the interface feels slow, reducing the number of running virtual devices or switching the graphics renderer can help distinguish application problems from environment limitations.

Testing on more than one AVD profile is useful because layout constraints, text scaling, and permission prompts can differ between Android versions. A small matrix of phone and tablet profiles provides better coverage than a single emulator.

5 Installation of Dart SDK in Windows

Dart is a programming language developed by Google, mainly used for building mobile, desktop, web, and server applications. The following steps describe how to install the Dart SDK on a Windows system.

1. Download the Dart SDK:
   - Go to the official Dart SDK website: https://dart.dev/get-dart.
   - Under the Windows section, choose the appropriate version (64-bit recommended) and download the ZIP file.
2. Extract the Dart SDK:
   - After downloading, extract the ZIP file to a preferred location, for example: C:\dart sdk
3. Set the Environment Variable:
   - Right-click on This PC → Properties → Advanced system settings.
   - Click on Environment Variables.
   - Under System variables, select Path → Edit.
   - Click New and add the path to the bin folder inside Dart SDK, e.g., C:\dart sdk\bin
4. Verify the Installation:
   - Open the Command Prompt.
   - Run the following command to verify that Dart is installed successfully:

```
dart --version
```

5. **Output:** The terminal should display the installed Dart version, for example:

```
Dart SDK version: 3.2.0 (stable)
```

Once the Dart SDK is installed and configured, you can use it to compile and run Dart programs or build Flutter applications.

The Dart SDK provides the Dart virtual machine, the compiler, the package manager, and the core libraries. Dart supports both just-in-time compilation for development and ahead-of-time compilation for release builds. The JIT mode enables hot reload and fast iteration, while AOT compilation produces optimised native code for production.

Dart uses a single-threaded event loop for most application code, with asynchronous operations scheduled as microtasks and events. Futures and async/await allow network requests and file operations to run without blocking the user interface. Isolates provide separate memory spaces for CPU-intensive work when true parallelism is required.

Null safety is a central feature of modern Dart. Variables are non-nullable by default, and the compiler requires developers to handle possible null values explicitly. This reduces common runtime errors and is especially useful when processing weather API responses, where fields may be missing or incomplete.

6 Installing Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful open-source code editor developed by Microsoft. It is widely used for Flutter and Dart development because of its excellent support for extensions, debugging, IntelliSense, and integration with the Flutter SDK. By installing VS Code, developers can write, test, and debug Flutter applications efficiently across multiple platforms.

Steps to Install VS Code and Set Up for Flutter Development

1. Download VS Code: Visit the official Visual Studio Code website at https://code.visualstudio.com/ and download the version suitable for your operating system (Windows, macOS, or Linux).
2. Install VS Code: Run the downloaded installer and complete the installation using the default settings. This ensures all necessary components, including PATH configuration, are properly set up.
3. Install Extensions: After launching VS Code, navigate to the Extensions panel using Ctrl + Shift + X. Search for and install the following essential extensions that are flutter, dart.
4. Create a New Flutter Project: Press Ctrl + Shift + P to open the Command Palette and search for Flutter: New Project. Select a project type (e.g., Application), specify a project name (e.g., quiz leaderboard), and choose a desired directory to create your new project.
5. Open the Project in VS Code: Once the project is created, VS Code automatically opens the folder. You can explore folders such as lib/, android/, and ios/ which are automatically generated by Flutter.

Visual Studio Code is a lightweight editor with an extensible architecture. Extensions add language support, debuggers, linters, formatters, and integrations with external tools. The editor's integrated terminal, source control view, and command palette make it possible to manage a Flutter project without switching between many applications.

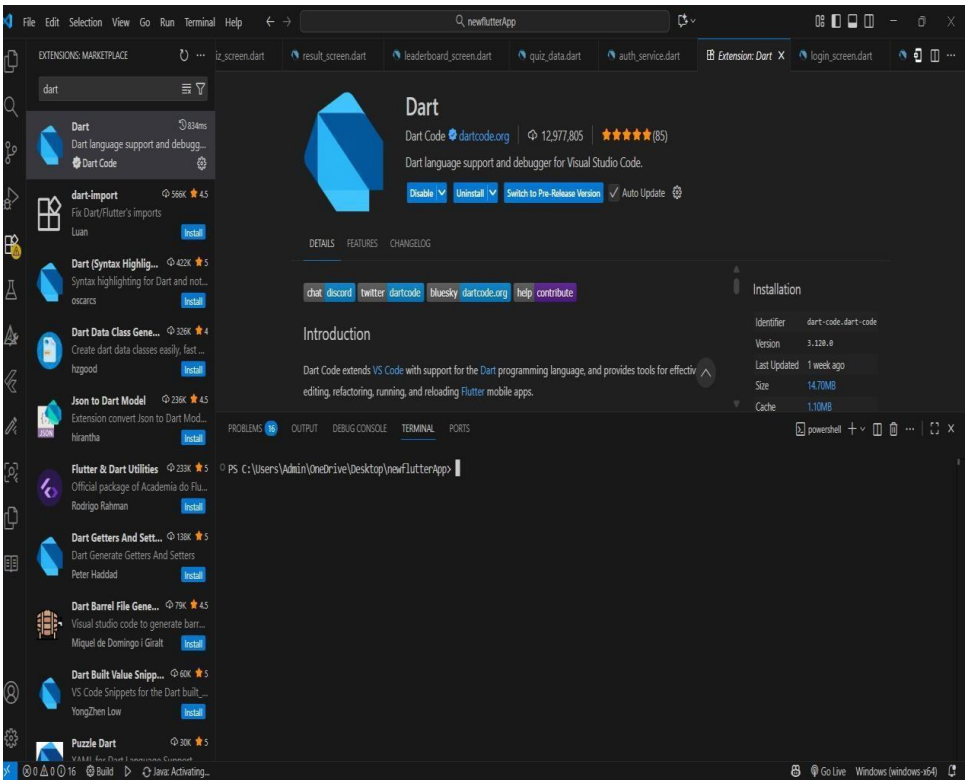
The Flutter and Dart extensions provide syntax highlighting, code completion, go-to-definition, refactoring, and widget snippets. They also expose commands for running the application, performing hot reload, opening DevTools, and inspecting the widget tree. These features shorten the edit-run-debug cycle and help developers understand how the interface is constructed.

For the Weather Alerts application, VS Code is useful for navigating between screens, widgets, and services. The debugger can pause execution while weather data is loaded, and the integrated terminal can run flutter pub get, flutter analyze, and flutter test. These tools support a disciplined development workflow.

VS Code settings such as format on save, rulers, and Dart analysis options can be shared through workspace configuration. Keeping these settings in the project helps every contributor use the same formatting and linting rules.

The integrated terminal is useful for running flutter doctor and flutter pub get without leaving the editor.

7 Creating a New Flutter Project



Installing the Dart and Flutter extensions in Visual Studio Code

Flutter is a framework developed by Google for building cross-platform applications using the Dart programming language. Before creating a new Flutter project, ensure that the Flutter SDK is installed and added to your system PATH.

Steps to Create a New Flutter Project

1. Open Command Prompt or Terminal.
2. Navigate to the desired folder:

```
cd C:\Users\<your-username>\Documents
```

3. Create a new Flutter project:

```
flutter create my_first_app
```

4. Navigate into the project directory:

```
cd my_first_app
```

5. Open the project in Visual Studio Code (or any preferred IDE):

```
code .
```

6. Run the Flutter application:

```
flutter run
```

7. Verify successful installation: Once the app runs, a default Flutter counter application appears on the emulator or connected device.

Project Structure

After running the above commands, Flutter automatically generates the following project structure:

- **lib/** Contains the main Dart source code.
- **android/, ios/, web/** Platform-specific code.
- **pubspec.yaml** Configuration file for dependencies and assets.
- **test/** Contains test files for unit and widget testing.
- **assets/** Directory to store images, fonts, and other static resources (added manually when needed).

The lib directory is the heart of a Flutter application. It contains the Dart code that defines the user interface, application logic, models, and services. A common organisation is to create separate folders for screens, widgets, models, services, and utilities, so that each file has a clear responsibility and can be tested independently.

The android and ios directories contain the native host projects. They define application permissions, launch screens, signing configuration, and platform-specific dependencies. The web directory contains the HTML entry point and web-specific configuration, while desktop directories provide the corresponding host projects for Windows, macOS, and Linux.

The test directory contains automated tests. Widget tests verify that interface components render and respond correctly, while unit tests verify pure logic such as weather code conversion or alert threshold evaluation. Keeping tests in a dedicated directory makes it easier to run them as part of a continuous integration workflow.

The pubspec.yaml file describes the project and its dependencies. It records the application name, version, Dart SDK constraint, package dependencies, development dependencies, and asset declarations. A well-maintained pubspec file ensures that every developer and build environment uses the same package versions.

For the Weather Alerts application, this structure supports a clear separation between presentation and data handling. Screens focus on layout and user interaction, widgets provide reusable visual components, services encapsulate API and authentication logic, and models represent weather and alert data. This separation makes the project easier to extend and maintain.

The generated project is a starting point rather than a fixed structure. As the application grows, feature folders can group screens, providers, models, and services by domain, while shared widgets and utilities remain in common directories.

Clear folder boundaries make code reviews easier and reduce accidental coupling between features.

8 Dart Basic Programming

Dart is a modern, object-oriented programming language developed by Google. It is optimized for building client applications, particularly mobile apps using the Flutter framework. Dart provides features like strong typing, asynchronous programming, and a rich standard library, making it both easy to learn and powerful for application development.

The following sections explain the fundamental programming concepts in Dart.

Variables

Variables are containers that hold data values. In Dart, every variable has a name, a type, and a value. Dart is a statically typed language, which means the data type of a variable is known at compile-time. However, Dart also supports type inference using the var keyword.

- **String:** Stores textual data.
- **int:** Stores whole numbers.
- **double:** Stores decimal numbers.
- **bool:** Stores Boolean values (true or false).
- **var:** Automatically infers the data type from the assigned
- **const:** Declares a compile-time constant — the value must be known and fixed at compile time.
- **final:** Declares a variable whose value can be set only once, but is determined at runtime.
- **Dart is type-safe,** meaning you cannot assign a value of a different type once declared.

Code:

```
void main() {
  String name = 'Lalithya';
  int age = 22;
  double height = 5.6;
  bool isStudent = true;
  print('Name: $name');
  print('Age: $age');
  print('Height: $height');
  print('Student: $isStudent');
}
```

Output:

```
Name: Lalithya
Age: 22
Height: 5.6
Student: true
```

Type inference is convenient for local variables, but public APIs benefit from explicit types and documentation.

Conditional Statements

Conditional statements are used to make decisions in a program. They allow different actions to be executed based on whether certain conditions are true or false. In Dart, conditional logic is implemented using if, else if, and else statements.

- **if:** Executes a block if the condition is true.
- **else if:** Checks another condition if the previous one is false.

Code:

```
void main() {
  int marks = 85;
  if (marks >= 90) {
    print('Grade: A+');
  } else if (marks >= 75) {
    print('Grade: A');
  } else {
    print('Grade: B');
  }
}
```

Output:

```
Grade: A
```

Loops

Loops are used to repeatedly execute a block of code as long as a specified condition is true. They help automate repetitive tasks and reduce redundancy. Dart supports the following types of loops:

- **for loop:** Repeats code a fixed number of times.
- **while loop:** Repeats while a condition is true.
- **do-while loop:** Executes code once, then checks the condition.

Code:

```
void main() {
  for (int i = 1; i <= 5; i++) {
    print('Iteration: $i');
  }
}
```

Output:

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
```

Control flow should be kept simple; deeply nested conditions usually indicate that a function should be split into smaller parts.

Lists

A List in Dart is an ordered collection of elements. Lists are similar to arrays in other programming languages. They can store multiple values of the same or different data types and allow random access using indexes.

Code:

```
void main() {
  List<String> fruits = ['Apple', 'Banana', 'Mango'];
  fruits.add('Orange');
  print('Fruits List: $fruits');
}
```

Output:

```
Fruits List: [Apple, Banana, Mango, Orange]
```

Maps

Maps in Dart are used to store data as key-value pairs, where each key is unique. They are similar to dictionaries in Python or hash maps in Java. Maps allow fast data retrieval using keys.

Code:

```
void main() {
  Map<String, int> scores = {'Alice': 90, 'Bob': 80, 'Charlie': 95};
  scores.forEach((name, mark) {
    print('$name scored $mark marks');
  });
}
```

Output:

```
Alice scored 90 marks Bob scored 80 marks Charlie scored 95 marks
```

Functions

Functions are reusable blocks of code designed to perform specific tasks. They help make programs modular and easier to maintain. In Dart, functions can take parameters and return values.

Code:

```
int addNumbers(int a, int b) {
  return a + b;
}

void main() {
  int result = addNumbers(5, 7);
  print('Sum: $result');
}
```

Output:

```
Sum: 12
```

9 Case Study on Weather alerts App

The Weather Alerts App is a cross-platform application developed using Flutter and Dart. It helps users monitor current weather conditions, view forecasts, and receive alerts about potentially dangerous weather conditions. This project demonstrates the practical application of Flutter UI design principles, responsive layouts, navigation, state management, REST API integration, Firebase authentication, and location-based services. In many areas, sudden weather changes and natural disasters such as heavy rainfall, thunderstorms, strong winds, and extreme temperatures can threaten people's safety. Users may not receive timely information about dangerous weather conditions, making it difficult to take appropriate precautions. The aim of this project is to develop a real-time weather monitoring and alert application that provides users with:

Key Features

- **User Authentication:** Secure login and registration functionality using Firebase Authentication.
- **Real-Time Weather Information:** Retrieves and displays current weather conditions, including temperature, humidity, wind speed, and weather descriptions.
- **Weather Forecasting:** Displays forecast information to help users understand expected weather changes.
- **Weather Alerts and Notifications:** Identifies potentially dangerous weather conditions and notifies users when configured alert thresholds or supported warning rules are met.
- **Location-Based Weather:** Uses the user's geographical location to retrieve relevant local weather information.
- **Interactive Map:** Displays location-related weather information and helps users understand local weather conditions.
- **Dashboard:** Provides a centralized view of current weather details, forecasts, and active weather alerts.
- **Profile Management:** Allows users to manage their personal information and application preferences.
- **Responsive UI:** Adapts the interface to different screen sizes and supports Android and web platforms.
- **Animations:** Provides smooth transitions and visual feedback to improve the user experience.
- **State Management:** Handles dynamic weather data, location updates, loading states, and alert information efficiently.

Technologies Used

- **Flutter SDK:** For building cross-platform UI and managing widgets.
- **Dart Language:** The core programming language for Flutter development.
- **VS Code:** Integrated development environment used for coding and debugging.
- **Android Emulator:** Used for testing and visualizing the app during development.
- **REST API:** For fetching and updating leaderboard data dynamically.
- **Provider Package:** For state management and real-time UI updates.
- **Firebase / Mock Backend:** For user authentication and data storage (if used).
- **GitHub:** For version control and project collaboration.

10 Exploring Flutter Widgets

Objective: To explore basic widgets like Text, Image, and Container.

Relation to Project: Used throughout the app for UI elements like alert cards and alert tiles.

Description: The Dashboard Widgets in the Weather Alerts app provide reusable UI component for displaying weather information in an attractive format. The GlassCard widget displays content using a glassmorphism effect with blur, borders, and rounded corners. The DashboardSectionHeader and DashboardPill widgets organize sections and display status labels or categories. The DashboardMetric widget presents important weather metrics with icons, values, and accent colors for easy understanding.

Widget Structure:

- The weather condition or alert title is displayed prominently in bold text.
- Weather details, such as temperature, humidity, and wind speed, are presented using styled metric widgets with icons and accent colors.
- Alert statuses are displayed using DashboardPill widgets, while DashboardSectionHeader organizes different sections of the dashboard.

Alerts card code:

```
import 'dart:ui';
import 'package:flutter/material.dart';
import '../theme/weather_palette.dart';

class GlassCard extends StatelessWidget {
  const GlassCard({
    super.key,
    required this.child,
    this.padding,
    this.margin,
    this.radius = WeatherPalette.radiusCard,
    this.fill,
    this.borderColor,
    this.blur = false,
    this.elevated = true,
    this.gradient,
    this.onTap,
  });

  final Widget child;
  final EdgeInsetsGeometry? padding, margin;
  final double radius;
  final Color? fill, borderColor;
  final bool blur, elevated;
  final Gradient? gradient;
  splashColor: WeatherPalette.cyan.withValues(alpha: 0.10),
  highlightColor:WeatherPalette.y
  an.withValues(alpha: 0.06),
      child: Padding(
        padding: padding ?? EdgeInsets.zero,
        child: child,
      ),
    );
```

```
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final shape = BorderRadius.circular(radius);

    Widget surface = DecoratedBox(
      decoration: BoxDecoration(
        color: gradient == null
            ? (fill ?? WeatherPalette.glassFill)
            : null,
        gradient: gradient,
        borderRadius: shape,
        border: Border.all(
          color: borderColor ?? WeatherPalette.border,
        ),
        boxShadow: elevated ? WeatherPalette.cardShadow : null,
      ),
      child: Padding(
        padding: padding ?? EdgeInsets.zero,
        child: child,
      ),
    );

    if (blur) {
      surface = ClipRRect(
        borderRadius: shape,
        child: BackdropFilter(
          filter: ImageFilter.blur(sigmaX: 18, sigmaY: 18),
          child: surface,
        ),
      );
    }

    if (onTap != null) {
      surface = Material(
        color: Colors.transparent,
        child: InkWell(
          onTap: onTap,
          borderRadius: shape,
          splashColor: WeatherPalette.cyan.withValues(alpha: 0.10),
          highlightColor: WeatherPalette.cyan.withValues(alpha: 0.06),
          child: surface,
        ),
      );
    }

    return margin == null

   class DashboardSectionHeader extends StatelessWidget
    { const DashboardSectionHeader({
          super.key,
          required
          this.title,
          this.icon,
          this.trailing,
    });
          final String
          title; final
          IconData? icon;
          final Widget?
```

```
        ? surface
        : Padding(padding: margin!, child: surface);
  }
}

class DashboardSectionHeader extends StatelessWidget {
  const DashboardSectionHeader({
    super.key,
    required this.title,
    this.icon,
    this.trailing,
  });
  final String title;
  final IconData? icon;
  final Widget? trailing;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Row(
      children: [
        if (icon != null) ...[
          Icon(icon, size: 18, color: WeatherPalette.cyan),
          const SizedBox(width: 8),
        ],
        Expanded(
          child: Text(
            title,
            style: theme.textTheme.titleSmall?.copyWith(
              color: WeatherPalette.textPrimary,
              fontWeight: FontWeight.w700,
              letterSpacing: 0.2,
            ),
          ),
        ),
        ?trailing,
  }
}
class DashboardPill extends StatelessWidget {
  const DashboardPill({
    super.key,
    required this.label,
    this.color = WeatherPalette.cyan,
    this.icon,
    this.filled = false,
  });
  final String label;
  final Color color;
  final IconData? icon;

      decoration: BoxDecoration(
        color: color.withValues(alpha: filled ? 0.18 : 0.09),
        borderRadius: BorderRadius.circular(
          WeatherPalette.radiusPill,
        ),
        border: Border.all(color: color.withValues(alpha: 0.42)),
      ),
    ),
  );
```

```
  final bool filled;
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(
        horizontal: 10,
        vertical: 5,
      ),
      decoration: BoxDecoration(
        color: color.withValues(alpha: filled ? 0.18 : 0.09),
        borderRadius: BorderRadius.circular(
          WeatherPalette.radiusPill,
        ),
        border: Border.all(color: color.withValues(alpha: 0.42)),
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          if (icon != null) ...[
            Icon(icon, size: 13, color: color),
            const SizedBox(width: 5),
          ],
          Flexible(
            child: Text(
              label,
              style: Theme.of(context).textTheme.labelSmall?.copyWith(
                    color: color,
                    fontWeight: FontWeight.w700,
                  ),
            ),
          ),
        ],
      ),
    );
  }
}
class DashboardMetric extends StatelessWidget {
  const DashboardMetric({
    super.key,
    required this.label,
    required this.value,
    this.icon,
    this.accent,
    this.valueStyle,
  });
  final String label, value;
  final IconData? icon;
  final Color? accent;
            Flexible(
              child: Text(
                label.toUpperCase(),
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style:
                  theme.textTheme.labelSmall?.copyWith(lette
                  rSpacing: 0.7,
                  fontSize: 10.5,
                ),
              ),
            ),
  );
```

```
  final TextStyle? valueStyle;
  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisSize: MainAxisSize.min,
      children: [
        Row(
          mainAxisSize: MainAxisSize.min,
          children: [
            if (icon != null) ...[
              Icon(
                icon,
                size: 13,
                color: accent ?? WeatherPalette.textMuted,
              ),
              const SizedBox(width: 5),
            ],
            Flexible(
              child: Text(
                label.toUpperCase(),
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: theme.textTheme.labelSmall?.copyWith(
                  color: WeatherPalette.textMuted,
                  fontWeight: FontWeight.w600,
                  letterSpacing: 0.7,
                  fontSize: 10.5,
                ),
              ),
            ),
          ],
        ),
        const SizedBox(height: 4),
        Text(
          value,
          maxLines: 2,
          overflow: TextOverflow.ellipsis,
          style: valueStyle ??
              theme.textTheme.titleSmall?.copyWith(
                color:  WeatherPalette.textPrimary,
                fontWeight: FontWeight.w700,
              ),
        ),
      ],
    );
  }
}
```

Output:



Weather Alerts Screen

Description: The WeatherCondition utility is a shared Flutter component used in the Weather Alerts application to display weather conditions consistently across different screens, such as the dashboard and alerts section. It identifies the human-readable weather condition, selects a suitable weather icon, and assigns an appropriate accent color based on the World Meteorological Organization (WMO) weather code.

Functionality: The utility provides three functions to represent weather conditions visually and textually. The weatherConditionLabel() function converts a WMO weather code into a readable condition name, such as Clear, Partly cloudy, Rain, Snow, or Thunderstorm. The weatherConditionIcon() function selects a corresponding Flutter Material icon for the weather condition. The weatherAccentFor() function assigns a suitable accent color to the icon based on the weather code.

Function Parameters: The utility contains the following three functions:

1. **weatherConditionLabel(int code):** Converts the WMO weather code into a human-readable weather condition name. It supports conditions such as clear skies, cloudy weather, fog, drizzle, rain, freezing rain, snow, rain showers, snow showers, and thunderstorms.
2. **weatherConditionIcon(int code):** Returns an appropriate Flutter IconData value for the specified weather condition. It uses Material icons such as Icons.wb_sunny_rounded, Icons.cloud_rounded, Icons.water_drop_rounded, and Icons.thunderstorm_rounded.
3. **weatherAccentFor(int code):** Returns a Color value to visually distinguish different weather conditions. It uses yellow for clear weather, blue tones for cloudy and rainy conditions, light cyan for snowy conditions, gray for fog, and orange for thunderstorms.

LeaderboardTile code:

```
import 'package:flutter/material.dart';

// Returns a readable label for a WMO weather code.
String weatherConditionLabel(int code) {
  if (code <= 1 && code >= 0) return 'Clear';
  if (code == 2) return 'Partly cloudy';
  if (code == 3) return 'Overcast';
  if (code == 45 || code == 48) {
    return code == 48 ? 'Rime fog' : 'Fog';
  }
  if (code >= 51 && code <= 57) return 'Drizzle';
  if (code >= 61 && code <= 67) {
    return code >= 66 ? 'Freezing rain' : 'Rain';
  }
  if (code >= 71 && code <= 77) return 'Snow';
  if (code >= 80 && code <= 82) return 'Rain showers';
  if (code == 85 || code == 86) return 'Snow showers';
  if (code >= 95) {

    return 'Unknown';

  }

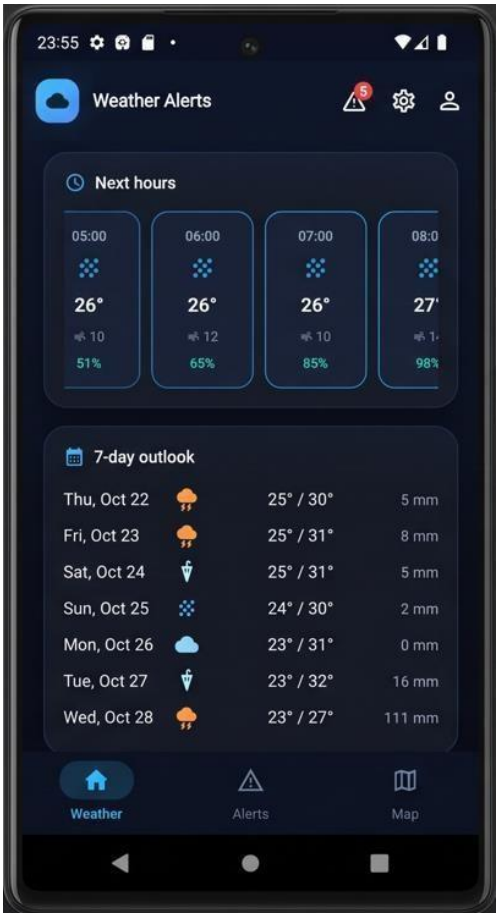
  if (code <= 1 && code >= 0) return 'Clear';
  if (code == 2) return 'Partly cloudy';
  if (code == 3) return 'Overcast';
  if (code == 45 || code == 48) {
```

```
    return code >= 96 ? 'Thunderstorm & hail' : 'Thunderstorm';
  }
  return 'Unknown';
}

// Returns an icon for the weather condition.
IconData weatherConditionIcon(int code) {
  if (code <= 1 && code >= 0) return Icons.wb_sunny_rounded;
  if (code == 2) return Icons.wb_cloudy_rounded;
  if (code == 3) return Icons.cloud_rounded;
  if (code == 45 || code == 48) return Icons.foggy;
  if (code >= 51 && code <= 57) return Icons.grain_rounded;
  if (code >= 61 && code <= 67) return Icons.water_drop_rounded;
  if (code >= 71 && code <= 77) return Icons.ac_unit_rounded;
  if (code >= 80 && code <= 82) return Icons.umbrella_rounded;
  if (code == 85 || code == 86) return Icons.ac_unit_rounded;
  if (code >= 95) return Icons.thunderstorm_rounded;
  return Icons.cloud_rounded;
}

// Returns an accent color for the weather icon.
Color weatherAccentFor(int code) {
  if (code <= 1 && code >= 0) return const Color(0xFFFBBF24);
  if (code == 2 || code == 3) return const Color(0xFF7DD3FC);
  if (code == 45 || code == 48) return const Color(0xFF94A3B8);
  if (code >= 51 && code <= 67) return const Color(0xFF38BDF8);
  if (code >= 71 && code <= 86) return const Color(0xFFA5F3FC);
  if (code >= 95) return const Color(0xFFFB923C);
  return const Color(0xFF7DD3FC);
}
```

Output:



Weekly weather report screen

11 Responsive UI Design

Objective: Design a responsive weather dashboard using Flutter widgets such as Row, Column, Expanded, and MediaQuery.

Relation to Project: The Weather Alerts application adapts its dashboard to different screen sizes and device orientations, displaying current weather information, weather conditions, and alerts clearly across mobile and web platforms.

Description: The DashboardScreen implements responsive UI design principles in Flutter to ensure that the weather dashboard automatically adapts to different screen sizes and orientations. Widgets such as SafeArea and SingleChildScrollView prevent content overflow and ensure that weather information remains accessible on smaller screens. The use of Row, Column, Expanded, and Container organizes weather information, including temperature, weather conditions, location details, and alert cards. MediaQuery helps determine screen dimensions and adjust padding, spacing, and layout according to the available screen size. The dashboard presents current weather details and weather alerts in a structured format. Flexible layout elements allow components to adjust their sizes and positions across mobile and web devices. Scrolling ensures that all weather information and alert details remain accessible on compact screens

Dashboard Screen code:

```
import 'dart:async';
import 'dart:math' as math;

import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../core/errors/app_exception.dart';
import '../../../core/utils/logger.dart';
import '../../../domain/entities/current_location.dart';
import '../../../services/firebase/device_token_providers.dart';
import '../../../services/weather_analysis/weather_analysis.dart';
import '../../alerts/application/alerts_providers.dart';
import '../../location/application/location_providers.dart';
import '../../location/application/location_state.dart';
import '../../weather/application/weather_providers.dart';
import '../../weather/application/weather_state.dart';
import 'dashboard_providers.dart';
import 'dashboard_state.dart';

/// Reload weather only after meaningful device movement.
const double kReloadOnMoveMeters = 500;

/// Coordinates location, weather analysis, alerts, and dashboard
state. class DashboardController extends Notifier<DashboardState> {
  DateTime? _lastIngestedAnalysisAt;
  CurrentLocation? _loadedLocation;

  @override
  DashboardState build() {
    ref.listen(weatherControllerProvider, (_, next) =>
    final granted = await ref.read(locationControllerProvider.notifier).
    requestPermission();
    if (!granted || !ref.mounted) return;
    await _captureAndLoad();
```

```
    syncFromWeather(next));
    ref.listen(locationControllerProvider, (_, next) { if (next.status ==
    LocationStatus.ready) _applyLatestFix(); });
    _bootstrap();
    return const DashboardState.bootstrapping();
  }

  /// Requests location permission and loads weather when
granted.
  Future<void> requestPermission() async {
    final granted = await ref.read(locationControllerProvider.notifier).
    requestPermission();
    if (!granted || !ref.mounted) return;
    await _captureAndLoad();
  }

  /// Refreshes the position and bypasses caches.
  Future<void> refresh() => _captureAndLoad(forceRefresh: true);

  /// Retries the weather pipeline after an error.
  Future<void> retry() => _captureAndLoad(forceRefresh: true);

  /// Opens system settings when location permission is denied
permanently.
  Future<void> openSettings() =>
      ref.read(locationControllerProvider.notifier).openSettings();

  Future<void> _bootstrap() async {
    final cached = await ref.read(resolveCurrentLocationProvider).lastKnown();
    if (!ref.mounted) return;
    if (cached != null) {
      _loadedLocation = cached;
      _set(state.copyWith(status: DashboardStatus.loading,
location: cached));
      await _loadWeather(cached);
    }
    await ref.read(locationControllerProvider.notifier).start();
    if (!ref.mounted) return;
    final location = ref.read(locationControllerProvider);
    if (location.position != null || location.status == LocationStatus.ready) {
      await _captureAndLoad();
    } else if (cached == null) {
      _set(const DashboardState(status: DashboardStatus.needsLocation));
    }
  }

  Future<void> _applyLatestFix() async {
    try {
      if (ref.read(locationControllerProvider).position == null)
return;
      await _captureAndLoad();
    } on AppException catch (e) {
      _set(state.copyWith(status: state.hasLocation ? state.status :
    _syncPushRouting();

    if (!forceRefresh && !_movedFarEnough(location)) return;
    await _loadWeather(location, forceRefresh: forceRefresh);
    final dx = (b.longitude - a.longitude) * math.pi / 180 * math.cos(meanLat);
    final dy = (b.latitude - a.latitude) * math.pi / 180;

      }
```

```
DashboardStatus.error, errorMessage: e.message));
    }
  }

  Future<void> _captureAndLoad({bool forceRefresh = false})
async {
    final resolver = ref.read(resolveCurrentLocationProvider);
    late final CurrentLocation location;
    try {
      location = await resolver.run();
    } on AppException catch (e) {
      _set(state.copyWith(status: state.hasLocation ? state.status :
    DashboardStatus.error, errorMessage: e.message));
      return;
    } catch (e) {
      ref.read(appLoggerProvider).warning('DashboardController',
'position capture failed: ${e.runtimeType}');
      _set(state.copyWith(status: state.hasLocation ? state.status :
    DashboardStatus.error, errorMessage: 'Could not determine your current
    location.'));
      return;
    }
    if (!ref.mounted) return;
    _set(state.copyWith(location: location, clearError: true));
    _syncPushRouting();
    if (!forceRefresh && !_movedFarEnough(location)) return;
    await _loadWeather(location, forceRefresh: forceRefresh);
  }

  void _syncPushRouting() {
    try {

ref.read(deviceTokenServiceProvider).syncCurrentLocation().igno  re();
    } catch (e) {
      ref.read(appLoggerProvider).debug('DashboardController',
'push routing sync skipped: ${e.runtimeType}');
    }
  }

  bool _movedFarEnough(CurrentLocation next) {
    final previous = _loadedLocation;
    return previous == null ||
        _distanceMeters(previous, next) >= kReloadOnMoveMeters;
  }

  static double _distanceMeters(CurrentLocation a,
CurrentLocation b) {
    const earthRadius = 6371000.0;
    final meanLat = (a.latitude + b.latitude) / 2 * math.pi / 180;
    final dx = (b.longitude - a.longitude) * math.pi / 180 * math.cos(meanLat);
    final dy = (b.latitude - a.latitude) * math.pi / 180;
    return math.sqrt(dx * dx + dy * dy) * earthRadius;
  }
    Future<void> _loadWeather( 28 await
    ref.read(weatherControllerProvider.notifier).load(locationKey:
    location.locationKey, latitude: location.latitude, longitude: location.
    longitude, forceRefresh: forceRefresh);
    if (ref.mounted) _loadedLocation = location;
  }
```

```
    CurrentLocation location, {
    bool forceRefresh = false,
  }) async {
    await ref.read(weatherControllerProvider.notifier).load(locationKey:
    location.locationKey, latitude: location.latitude, longitude: location.
    longitude, forceRefresh: forceRefresh);
    if (ref.mounted) _loadedLocation = location;
  }

  void _syncFromWeather(WeatherState weather) {
    if (!ref.mounted) return;
    final current = state;
    switch (weather.status) {
      case WeatherStatus.idle:
        return;
      case WeatherStatus.loading:
        _set(current.copyWith(status: DashboardStatus.loading,
bundle: weather.bundle));
      case WeatherStatus.ready:
        final analysis = ref.read(weatherAnalysisProvider);
        _set(DashboardState(status: DashboardStatus.ready,
location: current.location, bundle: weather.bundle, analysis: analysis ??
    current.analysis, activeAlerts: current.activeAlerts));
        if (analysis != null) _maybeIngest(analysis);
      case WeatherStatus.error:
        _set(current.copyWith(status: DashboardStatus.error,
bundle: weather.bundle, errorMessage: weather.errorMessage,
clearError: false));
    }
  }

  void _maybeIngest(WeatherAnalysis analysis) {
    final anchor = analysis.analyzedAtUtc;
    if (_lastIngestedAnalysisAt == anchor) return;
    _lastIngestedAnalysisAt = anchor;
    _ingest(analysis);
  }

  Future<void> _ingest(WeatherAnalysis analysis) async {
    final location = state.location;
    if (location == null) return;
    try {
      final active = await ref.read(alertIngestUseCaseProvider).ingest(analysis,
locationKey: location.locationKey);
      if (!ref.mounted) return;
      await ref.read(alertsControllerProvider.notifier).refresh();
      if (!ref.mounted) return;
      _set(state.copyWith(activeAlerts: active));
    } catch (e) {
      ref.read(appLoggerProvider).warning('DashboardController',
'alert ingest failed: ${e.runtimeType}');
    }
  }

  void _set(DashboardState next) {
    if (ref.mounted) state = next;
  }
}
```

Output:



Home Screen

12 Navigation Between Screens

Objective: To implement navigation using Navigator and named routes in the Weather Alerts application.

Relation to Project: Used to navigate between the Login, Dashboard, Weather Details, Alerts, and Profile screens.

Explanation:

- Users start at the LoginScreen, where they can log in or register for the application.
- After successful authentication, users are navigated to the DashboardScreen, which displays current weather information and weather alerts.
- From the DashboardScreen, users can:
  - Access the WeatherDetailsScreen to view detailed weather conditions and forecasts.
  - Access the AlertsScreen to view active weather alerts and safety recommendations.
  - Access the ProfileScreen to view or edit their profile information.
- When users log out from the dashboard:
  - They are redirected to the LoginScreen.
  - The navigation stack is cleared to prevent returning to authenticated screens using the back button

Sample Code:

```
void main() {
  runApp(MaterialApp(
    initialRoute: '/',
    routes: {
      '/': (context) => HomeScreen(),
      '/details': (context) => DetailsScreen(),
    },
  ));
}

// Navigate using: Navigator.pushNamed(context, '/details');
// Go back: Navigator.pop(context);
```

Navigation in Flutter is based on a stack of routes. Pushing a route places a new screen on top of the stack, popping removes the current screen, and replacing a route swaps one screen for another. Named routes provide a central place to define navigation paths, while route arguments allow data to be passed between screens.

The Weather Alerts application uses navigation to move from authentication screens to the dashboard and then to weather details, alerts, and profile screens. After a successful login, pushReplacement or pushAndRemoveUntil can be used so that the authentication screen is removed from the back stack and cannot be reached with the back button.

13 Stateful and Stateless Widgets

Objective: Understand widget lifecycles and state management in Flutter.

Relation to Project: Used in the Weather Alerts application to manage dynamic weather information, current location, weather alerts, and user interactions.

Description: In Flutter, widgets are the building blocks of the user interface and are classified as Stateless or Stateful depending on whether they maintain mutable state during their lifecycle.

Stateless Widgets:

Stateless widgets do not maintain mutable internal state. They display information based on the input parameters provided to them and rebuild when their parent updates them.

In the Weather Alerts application, examples include:

- **WeatherConditionDisplay:** Displays the weather condition using a label, icon, and accent color based on the weather code.
- **WeatherAlertTile:** Displays weather alert information, such as the alert title, description, and severity, when supplied through its parameters.

Stateless widgets are suitable for displaying weather information that depends on input data and does not require the widget itself to manage changing state.

Syntax for StatelessWidget:

```
class ClassName extends StatelessWidget {
  const ClassName({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      // UI code here
    );
  }
}
```

A stateless widget describes an interface that does not change after it is created. It is appropriate for static content, labels, icons, and presentation-only components. A stateful widget, by contrast, owns mutable state that can change during the widget's lifetime and rebuilds when setState is called.

The stateful widget lifecycle includes creation, insertion into the tree, updates, and disposal. The initState method is used for one-time initialisation, didUpdateWidget responds to configuration changes, and dispose releases controllers, listeners, and other resources. Understanding this lifecycle prevents memory leaks and stale callbacks.

As applications grow, local setState may become difficult to manage across many screens. State management solutions such as Provider or Riverpod centralise shared state and make dependencies explicit. The Weather Alerts application can use such a solution to share authentication, location, and weather data between screens.

Stateful Widgets:

Stateful widgets maintain state that can change during their lifecycle. They are useful for managing interactive elements and updating the interface when data changes.

In the Weather Alerts application, examples may include:

- **DashboardScreen:** Displays updated weather information and alerts.
- **LocationPermissionScreen:** Manages user interactions related to location permissions.

State management ensures that the dashboard reflects changes in weather conditions, location information, and active alerts. The application can use Flutter state management tools, such as Riverpod, to coordinate data updates and keep the interface synchronized with the underlying application state

Syntax for Stateful Widget:

```
class ClassName extends StatefulWidget {
  const ClassName({super.key});

  @override
  State<ClassName> createState() => _ClassNameState();
}

class _ClassNameState extends State<ClassName> {
  @override
  Widget build(BuildContext context) {
    return Container(
      // UI code here
    );
  }
}
```

Stateful widgets must handle asynchronous work carefully. When a network request completes, the widget may no longer be mounted, so the code should check the mounted property before calling setState. This prevents exceptions that occur when a response arrives after the user has navigated away from the screen.

Controllers such as TextEditingController, AnimationController, and ScrollController should be disposed when they are no longer needed. Listeners added to streams or providers should also be removed. These practices release memory and prevent callbacks from running after the associated interface has been destroyed.

For the Weather Alerts application, reliable state handling is essential because weather data, location updates, and authentication events arrive asynchronously. Clear ownership of state and careful cleanup make the application more stable and easier to debug.

Stateful widgets are appropriate when a screen owns short-lived interface state, such as a selected tab or a loading flag. Shared application state is better handled by a state management solution so that multiple screens observe the same source of truth.

Disposing controllers and subscriptions prevents memory leaks and duplicate callbacks when a screen is removed.

14 Custom Widgets and Styling

Objective: Create custom reusable widgets for the Weather Alerts application.

Relation to Project: Custom widgets are used to display weather conditions, current weather information, alert notifications, and reusable buttons across the Weather Alerts application's dashboard and other screens.

Description: The Weather Alerts application uses custom widgets and consistent styling to improve modularity, maintainability, and visual appearance. Custom widgets encapsulate reusable UI components that can be displayed across multiple screens, reducing code duplication and maintaining a uniform design.

Custom Widgets:

a. **WeatherConditionDisplay:** A reusable widget that displays the current weather condition using a descriptive label, a suitable weather icon, and an accent color based on the WMO weather code.
b. **WeatherAlertTile:** A reusable component that displays weather alert information, including the alert title, description, severity, and safety recommendations.
c. **WeatherInfoCard:** A reusable card widget that displays weather details such as temperature, humidity, wind speed, and precipitation.
d. **CustomButton:** A reusable button component with configurable text, icon, color, and onPressed action. It can be used for login, registration, refresh, and navigation actions.

Styling Approach:

e. Consistent color schemes using suitable weather-related colors to represent different weather conditions.
f. Rounded corners on weather cards, buttons, and other interface components.
g. Gradients and shadows applied to dashboard cards to improve visual appearance and hierarchy.
h. Clear and readable text styles for temperature, weather conditions, alert titles, and descriptions.
i. Consistent padding, margins, and spacing to maintain visual balance across different screen sizes.
j. Icons combined with text to help users quickly recognize weather conditions and alert types.

Benefits:

k. Improves code reusability through modular components.
l. Maintains a consistent appearance throughout the application.
m. Simplifies maintenance and the addition of new features.
n. Improves readability and usability of weather information.
o. Helps users identify dangerous weather conditions and safety alerts more easily.

Custom Button code:

```
import 'package:flutter/material.dart';

class WeatherSidebar extends StatefulWidget {
  const WeatherSidebar({super.key});

  @override
  State<WeatherSidebar> createState() => _WeatherSidebarState();
}

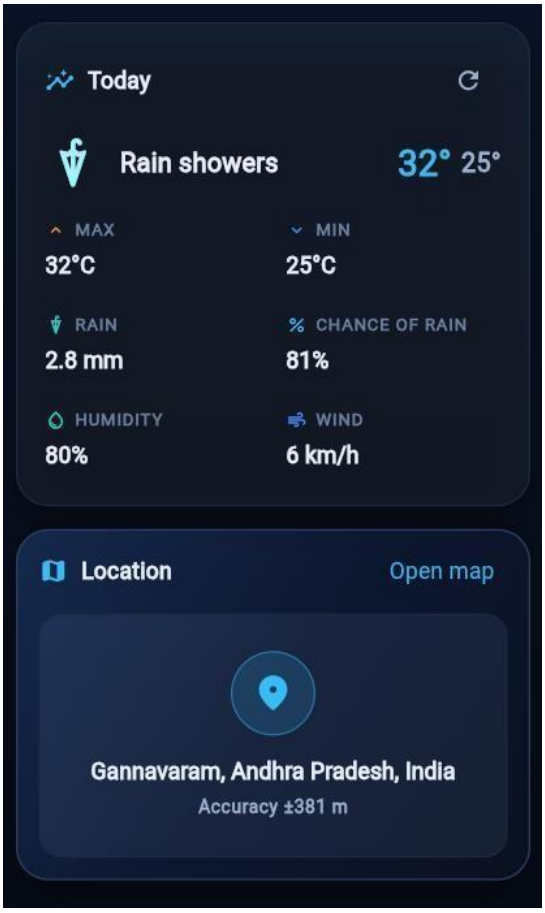
class _WeatherSidebarState extends State<WeatherSidebar> {
  String selected = 'Home';

  final items = [
    ('Home', Icons.grid_view_rounded),
    ('Hourly', Icons.access_time_rounded),
    ('Daily', Icons.calendar_month_rounded),
    ('Alerts', Icons.warning_amber_rounded),
    ('Map', Icons.map_rounded),
    ('Settings', Icons.settings_outlined),
  ];

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 255,
      color: const Color(0xFF091323),
      child: SafeArea(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Padding(
              padding: EdgeInsets.all(16),
              child: Text('MENU',
                  style: TextStyle(
                      color: Color(0xFF7186A4),
                      fontSize: 11,
                      fontWeight: FontWeight.bold)),
            ),
            ...items.map((item) {
              final active = selected == item.$1;
              return Padding(
                padding: const EdgeInsets.only(bottom: 5),
                child: InkWell(
                  onTap: () {
                    setState(() => selected = item.$1);
                    if (item.$1 == 'Home') {
                      Navigator.pushNamed(context, '/home');
                    }
                  },
                  child: AnimatedContainer(
                    duration: const Duration(milliseconds: 180),
                    height: 49,
                    margin: const EdgeInsets.symmetric(horizontal: 15),
                    padding: const EdgeInsets.symmetric(horizontal: 14),
                              Expanded(
                          child: Text(item.$1,
                              style: TextStyle(
                                  color: active
                                      ? const Color(0xFF35C8F5)
```

```
                    decoration: BoxDecoration(
                      color: active
                          ? const Color(0xFF102A3D)
                          : Colors.transparent,
                      borderRadius: BorderRadius.circular(15),
                      border: Border.all(
                        color: active
                            ? const Color(0xFF176080)
                            : Colors.transparent,
                      ),
                    ),
                    child: Row(
                      children: [
                        Icon(item.$2,
                            color: active
                                ? const Color(0xFF35C8F5)
                                : const Color(0xFFA4B7D2)),
                        const SizedBox(width: 15),
                        Expanded(
                          child: Text(item.$1,
                              style: TextStyle(
                                  color: active
                                      ? const Color(0xFF35C8F5)
                                      : const Color(0xFFA4B7D2),
                                  fontSize: 15,
                                  fontWeight: active
                                      ? FontWeight.w600
                                      : FontWeight.normal)),
                        ),
                        if (active)
                          Container(
                            width: 4,
                            height: 20,
                            decoration: BoxDecoration(
                              color: const Color(0xFF35C8F5),
                              borderRadius: BorderRadius.circular(5),
                            ),
                          ),
                      ],
                    ),
                  ),
                ),
              );
            }),
          ],
        ),
      ),
    );
  }
}
```

Output:



Location Widget

15 Forms and Validation

Objective: Create a form with validation in the Weather Alerts application.

Relation to Project: Used in the login and registration screens to authenticate users and collect their account details securely.

Description: The RegisterScreen implements a user registration form in the Weather Alerts application using Flutter form widgets to collect user information, such as name, email, password, and confirm password. It demonstrates input validation, error handling, and user feedback to ensure that users provide valid information before creating an account.

Input Validation:

a. Checks that all required fields are completed before submission.
b. Validates the email format to ensure the user enters a valid email address.
c. Ensures that the password and confirm password fields match.
d. Displays appropriate error messages when validation fails.

User Feedback:

e. Displays a CircularProgressIndicator while registration is in progress.
f. Shows success or error messages using SnackBar.
g. Prevents repeated submissions while registration is processing.

Navigation:

h. After successful registration, the user is navigated to the DashboardScreen.
i. Uses Navigator.pushAndRemoveUntil() to clear the navigation stack when required.
j. Provides a Back to Login option for existing users.

Code Structure:

k. Uses setState() or an appropriate state management solution to update the interface dynamically. Uses TextEditingController to manage user input.
l. Uses Form and TextFormField widgets to collect and validate registration details.
m. The register() method handles validation, loading state, and communication with the authentication service.

Flutter forms use a Form widget with a GlobalKey<FormState> to coordinate validation across multiple fields. Each TextFormField provides a validator callback that returns an error message when the input is invalid. The form's validate method runs all validators and reports whether the input can be submitted.

Input formatters restrict what the user can type, while keyboard types and text input actions improve the editing experience. AutovalidateMode can validate fields as the user types or after the first submission attempt. These features reduce frustration by providing immediate, specific feedback.

Registration Screen Code:

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();
  final _name = TextEditingController();
  final _email = TextEditingController();
  final _phone = TextEditingController();
  final _password = TextEditingController();
  final _confirm = TextEditingController();
  bool _loading = false, _hide = true;

  @override
  void dispose() {
    _name.dispose();
    _email.dispose();
    _phone.dispose();
    _password.dispose();
    _confirm.dispose();
    super.dispose();
  }

  String? requiredField(String? v) =>
      v == null || v.trim().isEmpty ? 'This field is required' : null;

  Widget field(String label, TextEditingController c,
      {TextInputType type = TextInputType.text,
      String? Function(String?)? validator,
      bool password = false}) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 14),
      child: TextFormField(
        controller: c,
        keyboardType: type,
        obscureText: password && _hide,
        validator: validator ?? requiredField,
        decoration: InputDecoration(
          labelText: label,
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(12),
          ),
          prefixIcon: Icon(password ? Icons.lock_outline : Icons.person_outline),
          suffixIcon: password
              ? IconButton(
                  icon: Icon(_hide ? Icons.visibility_off : Icons.visibility),
                  onPressed: () => setState(() => _hide = !_hide),
                )
                 TextButton(
                   onPressed: () => Navigator.pop(context),
                   child: const Text('Already registered? Login'),
                 ),
```

```
              : null,
        ),
      ),
    );
  }

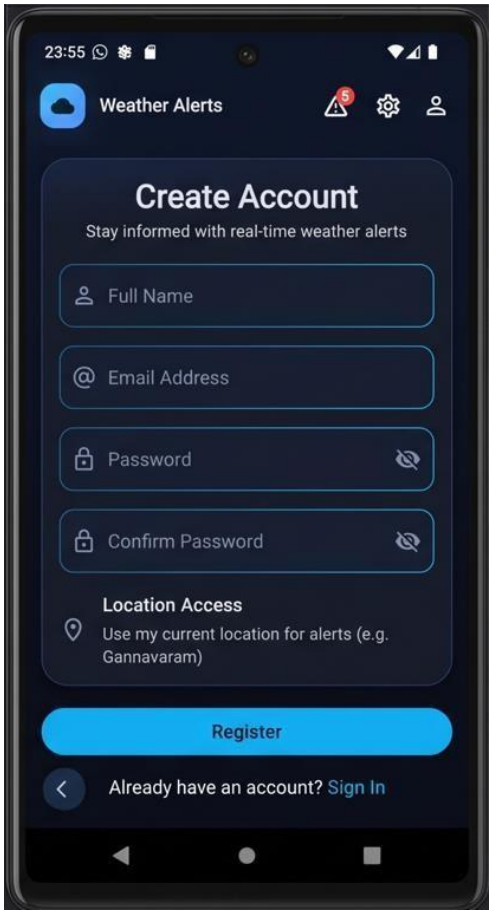
  Future<void> register() async {
    if (!_formKey.currentState!.validate()) return;
    setState(() => _loading = true);
    try {
      final result = await FirebaseAuth.instance.createUserWithEmailAndPassword(
        email: _email.text.trim(),
        password: _password.text,
      );
      await result.user?.updateDisplayName(_name.text.trim());
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Registration successful!')),
      );
      Navigator.pushNamedAndRemoveUntil(context, '/home', (_) => false);
    } on FirebaseAuthException catch (e) {
      if (!mounted) return;
      final message = switch (e.code) {
        'email-already-in-use' => 'Email already registered',
        'weak-password' => 'Choose a stronger password',
        'invalid-email' => 'Invalid email address',
        _ => e.message ?? 'Registration failed',
      };
      ScaffoldMessenger.of(context)
          .showSnackBar(SnackBar(content: Text(message)));
    } catch (_) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Something went wrong')),
      );
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Create Account')),
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: ConstrainedBox(
              constraints: const BoxConstraints(maxWidth: 450),
              child: Form(
                key: _formKey,
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    const Icon(Icons.cloud, size: 60, color: Colors.lightBlue),

                   const SizedBox(height: 8),
                   ElevatedButton(
                     onPressed: _loading ? null : register,
                     child: _loading
```

```
                    const Text('Join Weather Alerts',
                        textAlign: TextAlign.center,
                        style: TextStyle(
                            fontSize: 25, fontWeight: FontWeight.bold)),
                    const SizedBox(height: 24),
                    field('Full Name', _name),
                    field('Email Address', _email,
                        type: TextInputType.emailAddress,
                        validator: (v) =>
                            v != null &&
                                    RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$')
                                        .hasMatch(v.trim())
                                ? null
                                : 'Enter a valid email'),
                    field('Phone Number', _phone,
                        type: TextInputType.phone,
                        validator: (v) => v != null &&
                                RegExp(r'^\d{10}$').hasMatch(v.trim())
                            ? null
                            : 'Enter a 10-digit phone number'),
                    field('Password', _password, password: true,
                        validator: (v) =>
                            v != null && v.length >= 8
                                ? null
                                : 'Minimum 8 characters'),
                    field('Confirm Password', _confirm,
                        password: true,
                        validator: (v) => v == _password.text
                            ? null
                            : 'Passwords do not match'),
                    const SizedBox(height: 8),
                    ElevatedButton(
                      onPressed: _loading ? null : register,
                      child: _loading
                          ? const CircularProgressIndicator()
                          : const Text('Register'),
                    ),
                    TextButton(
                      onPressed: () => Navigator.pop(context),
                      child: const Text('Already registered? Login'),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

Output:



Registration Screen

Description: The LoginScreen of the Weather Alerts application allows registered users to securely log in using their email address and password. After successful authentication through Firebase Authentication, users are navigated to the DashboardScreen, where they can view current weather conditions and weather alerts.

Input Validation:

- Checks that the email address and password fields are not empty before submission.
- Validates the email format and displays appropriate error messages when the input is invalid.
- Displays an error message if authentication fails, such as when the credentials are incorrect.

User Feedback:

- Shows a CircularProgressIndicator while the login request is being processed.
- Displays error messages for invalid credentials, network issues, or authentication failures.
- Provides feedback when login is successful.

Navigation:

- On successful login, the user is navigated to the DashboardScreen using Navigator.pushAndRemoveUntil() or Navigator.pushNamedAndRemoveUntil() to clear the navigation stack and prevent returning to the login screen using the Back button.
- Provides a “Create an Account” button that navigates to the RegisterScreen for new user registration.

Code Structure:

- Uses TextEditingController to manage email and password input.
- The login() method handles input validation, loading-state management, and calls Firebase Authentication to verify the user's credentials.
- Error messages and loading indicators are updated using setState(), demonstrating the use of a StatefulWidget for form handling.
- Integrates Firebase Authentication to manage user login securely and provide access to the Weather Alerts dashboard after successful authentication.

Login Screen code:

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _email = TextEditingController();
  final _password = TextEditingController();
```

```
  bool _loading = false, _hide = true;

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

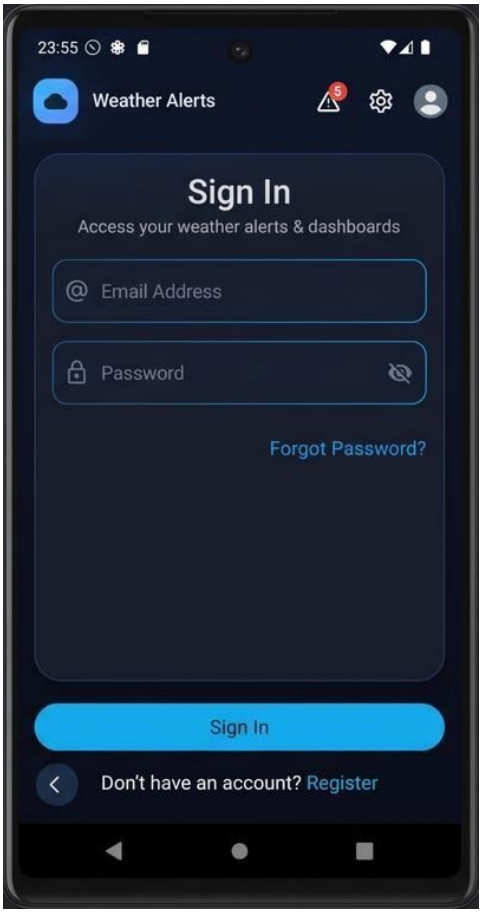
  Future<void> login() async {
    if (!_formKey.currentState!.validate()) return;
    setState(() => _loading = true);
    try {
      await FirebaseAuth.instance.signInWithEmailAndPassword(
        email: _email.text.trim(),
        password: _password.text,
      );

      if (!mounted) return;
      Navigator.pushNamedAndRemoveUntil(context, '/home', (_) => false);
    } on FirebaseAuthException catch (e) {
      if (!mounted) return;
      final msg = switch (e.code) {
        'user-not-found' || 'invalid-credential' =>
          'Invalid email or password',
        'wrong-password' => 'Incorrect password',
        'invalid-email' => 'Enter a valid email',
        'network-request-failed' => 'Check your internet connection',
        _ => e.message ?? 'Login failed',
      };
      ScaffoldMmessenger.of(context)
          .showSnackBar(SnackBar(content: Text(msg)));
    } catch (_) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Something went wrong')),
        );
      }
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }


  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Login')),
    body: Center(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(24),
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 450),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                const Icon(Icons.cloud, size: 60, color: Colors.lightBlue),
                const Text('Weather Alerts',
                labelText: 'Password',
                prefixIcon: const Icon(Icons.lock_outline), border: const
                OutlineInputBorder(),
```

```
                    textAlign: TextAlign.center,
                    style: TextStyle(fontSize: 25,
                        fontWeight: FontWeight.bold)),
                const SizedBox(height: 24),
                TextFormField(
                  controller: _email,
                  keyboardType: TextInputType.emailAddress,
                  decoration: const InputDecoration(
                    labelText: 'Email Address',
                    prefixIcon: Icon(Icons.email_outlined),
                    border: OutlineInputBorder(),
                  ),
                  validator: (v) => v != null &&
                      RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$')
                          .hasMatch(v.trim())
                      ? null : 'Enter a valid email',
                ),
                const SizedBox(height: 16),
                TextFormField(
                  controller: _password,
                  obscureText: _hide,
                  decoration: InputDecoration(
                    labelText: 'Password',
                    prefixIcon: const Icon(Icons.lock_outline),
                    border: const OutlineInputBorder(),
                    suffixIcon: IconButton(
                      icon: Icon(_hide
                          ? Icons.visibility_off : Icons.visibility),
                      onPressed: () => setState(() => _hide = !_hide),
                    ),
                  ),
                  validator: (v) => v == null || v.isEmpty
                      ? 'Enter your password' : null,
                ),
                const SizedBox(height: 24),
                ElevatedButton(
                  onPressed: _loading ? null : login,
                  child: _loading
                      ? const CircularProgressIndicator()
                      : const Text('Login'),
                ),
                TextButton(
                  onPressed: () => Navigator.pushNamed(context, '/register'),
                  child: const Text('Create an Account'),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
}
```

Output:



Login Screen

16 Animations

Objective: Add basic UI animations to improve the user experience of the Weather Alerts application.

Relation to Project: Used to provide smooth transitions between screens and visually highlight weather information, alert cards, and loading states.

Description: The Flutter Weather Alerts application incorporates animations to make the interface smoother, more interactive, and easier to understand. Animations enhance the dashboard, weather information, alert notifications, and navigation between screens.

Weather Dashboard Animation:

a. Uses Flutter animation widgets such as AnimatedContainer and TweenAnimationBuilder to create smooth visual transitions for weather information.

b. Displays weather details and dashboard cards with subtle entrance animations.

c. Helps users easily notice changes in weather conditions and important information.

Weather Alert Card Animation:

d. Uses AnimatedContainer or TweenAnimationBuilder to animate alert cards when their appearance or alert status changes.

e. Highlights dangerous weather alerts using visual changes such as color transitions, making important warnings easier to notice.

f. Improves the visibility of severe weather warnings without distracting users from essential information.

Implicit Animations for UI Feedback:

g. Uses CircularProgressIndicator on login, registration, and weather-loading screens to indicate ongoing operations.

h. Uses animated widgets to provide smooth feedback when weather data is refreshed or dashboard content changes.

i. Helps users understand that the application is processing requests or retrieving real-time weather information.

Navigation Transitions:

j. Uses Flutter's default MaterialPageRoute transitions or named-route navigation to provide smooth transitions between screens.

k. Supports navigation between LoginScreen, RegisterScreen, DashboardScreen, Weather Details, Alerts, and Map screens.

l. Provides a consistent navigation experience and improves the overall usability of the application.

Animations should support understanding rather than distract from it. Short durations, consistent easing curves, and meaningful transitions help users follow changes in weather conditions, alert status, and navigation. Reduced-motion settings should be respected for users who prefer less movement.

Animations:

```
import 'package:flutter/material.dart';
import 'weather_palette.dart';

abstract final class AppTheme {
  AppTheme._();
  static const Color seedColor = Color(0xFF38BDF8);
  static const Color safetyGreen = WeatherPalette.safe;

  static ThemeData light() => _build(Brightness.light);
  static ThemeData dark() => _build(Brightness.dark);
  static Color canvasFor(Brightness brightness) =>
      brightness == Brightness.dark
          ? WeatherPalette.canvas
          : const Color(0xFFF4F7FC);

  static ThemeData _build(Brightness brightness) {
    final scheme = _schemeFor(brightness);
    final textTheme = _textThemeFor(scheme, brightness);
    return ThemeData(
      useMaterial3: true,
      colorScheme: scheme,
      brightness: brightness,
      textTheme: textTheme,
      scaffoldBackgroundColor: canvasFor(brightness),
      canvasColor: canvasFor(brightness),
      appBarTheme: AppBarTheme(
        centerTitle: false,
        backgroundColor: brightness == Brightness.dark
            ? WeatherPalette.canvasElevated
            : scheme.surface,
        foregroundColor: scheme.onSurface,
        elevation: 0,
        scrolledUnderElevation: 0,
        surfaceTintColor: Colors.transparent,
      ),
      cardTheme: CardThemeData(
        elevation: 0,
        margin: EdgeInsets.zero,
        color: brightness == Brightness.dark
            ? WeatherPalette.surface
            : scheme.surfaceContainerLowest,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(WeatherPalette.radiusCard),
        ),
        clipBehavior: Clip.antiAlias,
      ),
      dividerTheme: DividerThemeData(
        color: scheme.outlineVariant.withValues(alpha: 0.4),
        space: 1,
        thickness: 1,
      ),

      chipTheme: ChipThemeData(
              color: states.contains(WidgetState.selected)
                  ? WeatherPalette.cyan
                  : WeatherPalette.textMuted,
            ),
          ),
```

```
        backgroundColor: WeatherPalette.surfaceRaised,
        side: BorderSide(color: WeatherPalette.border),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(WeatherPalette.radiusPill),
        ),
        labelStyle: textTheme.labelSmall,
      ),
      navigationBarTheme: NavigationBarThemeData(
        backgroundColor: WeatherPalette.canvasElevated,
        surfaceTintColor: Colors.transparent,
        indicatorColor: WeatherPalette.cyan.withValues(alpha: 0.16),
        height: 68,
        labelBehavior: NavigationDestinationLabelBehavior.alwaysShow,
        iconTheme: WidgetStateProperty.resolveWith(
          (states) => IconThemeData(
            size: 24,
            color: states.contains(WidgetState.selected)
                ? WeatherPalette.cyan
                : WeatherPalette.textMuted,
          ),
        ),
        labelTextStyle: WidgetStateProperty.resolveWith(
          (states) => textTheme.labelSmall?.copyWith(
            fontWeight: states.contains(WidgetState.selected)
                ? FontWeight.w700
                : FontWeight.w500,
            color: states.contains(WidgetState.selected)
                ? WeatherPalette.cyan
                : WeatherPalette.textMuted,
          ),
        ),
      ),
      filledButtonTheme: FilledButtonThemeData(
        style: FilledButton.styleFrom(
          minimumSize: const Size(64, 48),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(WeatherPalette.radiusControl),
          ),
        ),
      ),
      outlinedButtonTheme: OutlinedButtonThemeData(
        style: OutlinedButton.styleFrom(
          minimumSize: const Size(64, 48),
          side: const BorderSide(color: WeatherPalette.borderStrong),
          foregroundColor: WeatherPalette.textPrimary,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(WeatherPalette.radiusControl),
          ),
        ),
      ),
      textButtonTheme: TextButtonThemeData(
        style: TextButton.styleFrom(minimumSize: const Size(48, 40)),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: WeatherPalette.surface,
        contentPadding: const EdgeInsets.symmetric(
          horizontal: 16,
          vertical: 14,
        ),
```

```
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(WeatherPalette.radiusControl),
          borderSide: const BorderSide(color: WeatherPalette.border),
        ),

        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(WeatherPalette.radiusControl),
          borderSide: const BorderSide(color: WeatherPalette.border),
        ),
        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(WeatherPalette.radiusControl),
          borderSide: const BorderSide(
            color: WeatherPalette.cyan,
            width: 1.6,
          ),
        ),

        contentPadding: const EdgeInsets.symmetric(
          horizontal: 16,
          vertical: 14,
        ),
      ),

      snackBarTheme: SnackBarThemeData(
        behavior: SnackBarBehavior.floating,
        backgroundColor: WeatherPalette.surfaceRaised,
        contentTextStyle: textTheme.bodyMedium,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(WeatherPalette.radiusControl),
        ),
      ),

      tooltipTheme: TooltipThemeData(
        decoration: BoxDecoration(
          color: WeatherPalette.surfaceRaised,
          borderRadius: BorderRadius.circular(WeatherPalette.radiusControl),
          border: Border.all(color: WeatherPalette.border),
        ),
        textStyle: textTheme.bodySmall,
      ),
      progressIndicatorTheme: ProgressIndicatorThemeData(
        color: WeatherPalette.cyan,
        linearTrackColor: WeatherPalette.surfaceRaised,
        circularTrackColor: WeatherPalette.surfaceRaised,
      ),
    );
  }

  static ColorScheme _schemeFor(Brightness brightness) {
    if (brightness == Brightness.light) {
      return ColorScheme.fromSeed(
        seedColor: seedColor,
        brightness: Brightness.light,
        dynamicSchemeVariant: DynamicSchemeVariant.fidelity,
      ).copyWith(surface: const Color(0xFFF4F7FC));
    }

    return ColorScheme.fromSeed(
      seedColor: seedColor,
      brightness: Brightness.dark,
      dynamicSchemeVariant: DynamicSchemeVariant.fidelity,
```

```
    ).copyWith(
      primary: WeatherPalette.cyan,
      onPrimary: const Color(0xFF04202E),
      secondary: WeatherPalette.blue,
      onSecondary: Colors.white,
      tertiary: WeatherPalette.teal,
      error: WeatherPalette.danger,
      onError: Colors.white,
      surface: WeatherPalette.canvas,
      onSurface: WeatherPalette.textPrimary,
      surfaceContainerLowest: WeatherPalette.canvas,
      surfaceContainerLow: WeatherPalette.canvasElevated,
      surfaceContainer: WeatherPalette.surface,
      surfaceContainerHigh: WeatherPalette.surfaceRaised,
      surfaceContainerHighest: WeatherPalette.surfaceRaised,
      onSurfaceVariant: WeatherPalette.textSecondary,
      outline: WeatherPalette.borderStrong,
      outlineVariant: WeatherPalette.border,
    );
  }
  static TextTheme _textThemeFor(
    ColorScheme scheme,
    Brightness brightness,
  ) {
    final base = Typography.material2021(colorScheme: scheme).englishLike.apply(
      bodyColor: scheme.onSurface,
      displayColor: scheme.onSurface,
    );
    return base.copyWith(
      displayMedium: base.displayMedium?.copyWith(
        fontWeight: FontWeight.w300,
        letterSpacing: -2,
        height: 1.05,
      ),
      displaySmall: base.displaySmall?.copyWith(
        fontWeight: FontWeight.w400,
        letterSpacing: -1,
      ),
      headlineSmall: base.headlineSmall?.copyWith(
        fontWeight: FontWeight.w700,
        letterSpacing: -0.4,
      ),
      titleLarge: base.titleLarge?.copyWith(fontWeight: FontWeight.w700),
      titleMedium: base.titleMedium?.copyWith(fontWeight: FontWeight.w600),
      titleSmall: base.titleSmall?.copyWith(fontWeight: FontWeight.w600),
      labelSmall: base.labelSmall?.copyWith(fontWeight: FontWeight.w600),
      bodyMedium: base.bodyMedium?.copyWith(height: 1.45),
      bodySmall: base.bodySmall?.copyWith(
        height: 1.4,
        color: brightness == Brightness.dark
            ? WeatherPalette.textSecondary
            : scheme.onSurfaceVariant,
      ),
    );
  }
}
```

Output:



Map Screen

17 REST API Integration

Objective: Fetch real-time weather data from an external API and display it in the application.

Relation to Project: Used in the Weather Alerts dashboard to retrieve current weather conditions, forecast information, and weather data for generating alerts.

Description: The Weather Alerts application uses an HTTP GET request to fetch weather information from an external weather API, such as OpenWeatherMap or WeatherAPI. The retrieved JSON response is processed and displayed on the dashboard.

API Data Fetching:

a. Sends an HTTP GET request to the weather API using the user's current geographical coordinates.
b. Retrieves temperature, humidity, wind speed, weather conditions, and forecast information.
c. Converts the JSON response into usable data for the Flutter interface.

Displaying Weather Information:

d. Displays the fetched weather information on the DashboardScreen.
e. Updates weather details when the user changes location or refreshes the data.
f. Handles loading states and API errors.

Code:

```
final response = await http.get(
  Uri.parse(
    'https://api.openweathermap.org/data/2.5/weather'
    '?lat=$latitude&lon=$longitude&appid=$apiKey&units=metric',
  ),
);

if (response.statusCode == 200) {
  final weatherData = jsonDecode(response.body);
  print('Temperature: ${weatherData['main']['temp']}°C');
} else {
  throw Exception('Failed to fetch weather data');
}
```

REST APIs expose resources through HTTP methods such as GET, POST, PUT, and DELETE. A weather API typically returns JSON containing current conditions, forecasts, and metadata. The client sends a request with the required parameters, checks the status code, and decodes the response into application models.

Robust API integration includes timeouts, retry policies, and error handling for network failures, invalid responses, and rate limits. API keys should not be hard-coded in public repositories; they should be stored in environment configuration or a secure backend. These practices protect both the application and the service provider.

18 Testing and Debugging

Objective: To perform UI testing, unit testing, and use debugging tools to ensure the Weather Alerts application functions correctly.

Explanation: Testing is the process of verifying that the Weather Alerts application works as intended. Testing ensures that users can register, log in, view current weather conditions, receive dangerous weather alerts, and access location-based forecasts without errors.

Types of testing include:

a. **Unit Testing:** Tests individual functions or classes. Example: Checking whether the weather analysis function correctly identifies dangerous weather conditions and generates the appropriate alert.
b. **Widget Testing:** Verifies that UI components display correctly and respond to user interactions. Example: Checking whether the DashboardScreen displays temperature, humidity, wind speed, and weather alert cards correctly.
c. **Integration Testing:** Tests the complete application workflow, including Firebase authentication, weather API calls, and notification services. Example: Login → Fetch Current Weather → Analyze Weather Conditions → Generate Alert → Send Notification.
d. **Manual Testing:** Running the application on a physical device, emulator, or web browser to test user interactions directly. Example: Verifying that the application detects the user's location, refreshes weather information, and displays appropriate alerts during dangerous weather conditions.

Debugging:

Debugging involves identifying and fixing errors in the user interface, weather API requests, Firebase authentication, location services, and notification delivery. Tools used include Flutter DevTools, print() or logging statements, try-catch blocks, and Postman for testing weather API endpoints.

Testing verifies that an application behaves as expected and continues to do so as it changes. Unit tests check individual functions and models, widget tests verify interface components, and integration tests exercise complete workflows. Together these levels form a testing pyramid that balances speed, confidence, and maintenance cost.

Debugging complements testing by helping developers locate the cause of a failure. Flutter DevTools provides widget inspection, performance profiling, network inspection, and logging. Breakpoints, print statements, and structured error handling can be used to understand the state of the application at the moment a problem occurs.

Automated tests should cover critical paths such as login, registration, weather loading, alert generation, and navigation. Running tests regularly, ideally as part of a continuous integration pipeline, catches regressions early and makes refactoring safer.

Sample UI Test Code:

```
testWidgets('Login Screen loads', (tester) async {
  await tester.pumpWidget(MyApp());
  expect(find.text('Login'), findsOneWidget);
});
```

Sample Debugging Example:

```
try {
  final token = await authService.login(email, password);
  if (token == null) {
    print('Login failed: Invalid response');
  }
} catch (e) {
  print('Login error: $e');
}
```

The sample UI test uses testWidgets to create a controlled test environment. WidgetTester pumps the application, builds the widget tree, and allows the test to interact with the interface. Finders locate widgets by text, type, or key, and expectations verify that the visible state matches the intended behaviour.

Widget tests are fast because they run without a real device, but they do not replace integration testing. They are most useful for verifying that a screen renders, that validation messages appear, and that user interactions trigger the correct callbacks.

The sample debugging example shows how try-catch blocks and logging can make failures visible. When an authentication request fails, the application can record the error, show a user-friendly message, and avoid leaving the interface in an inconsistent state.

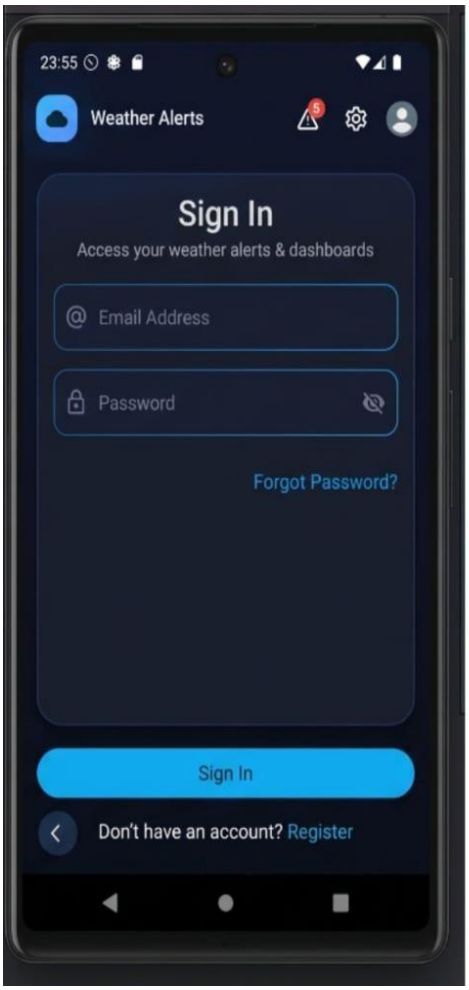
A practical testing strategy combines automated tests with manual checks on emulators and physical devices. This combination catches both logic errors and platform-specific issues such as permission prompts, keyboard behaviour, and rendering differences.

A balanced testing strategy follows the testing pyramid: many fast unit tests, fewer widget tests, and a small number of integration tests that exercise complete workflows. This distribution provides quick feedback while still covering the interactions that are most likely to break.

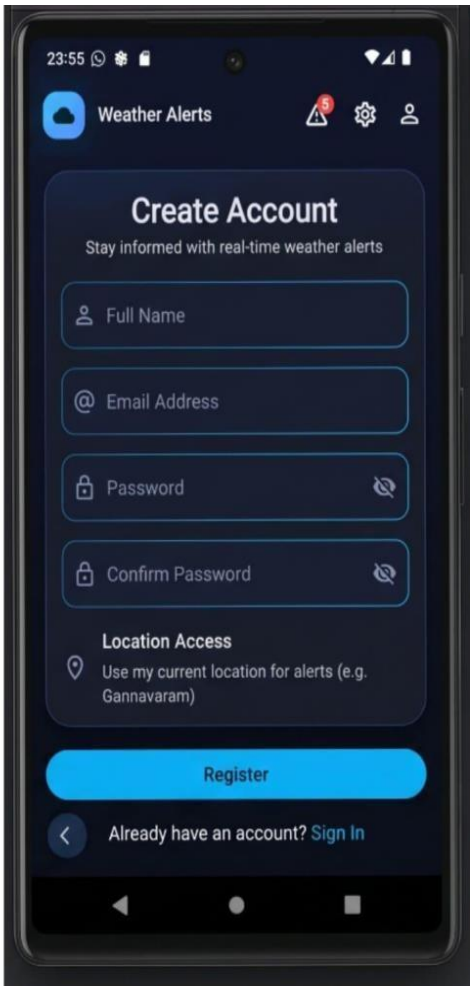
Continuous integration can run formatting checks, static analysis, and automated tests whenever code is pushed. Catching failures early reduces the cost of fixing them and helps keep the main branch stable.

Debugging should move from evidence to hypothesis rather than guessing. Logs, breakpoints, widget inspection, and network traces help identify whether a problem originates in the interface, state management, data parsing, or the backend.

19 Work Flow Of Screens:



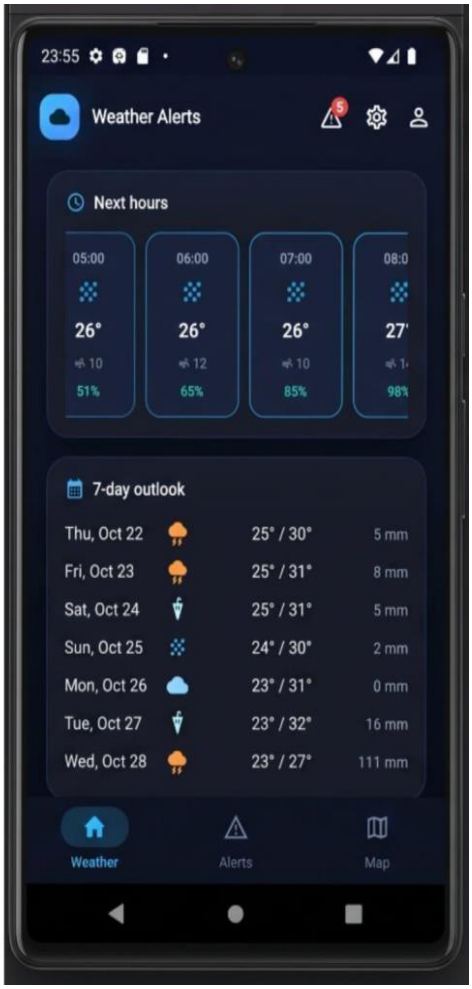
Login Screen



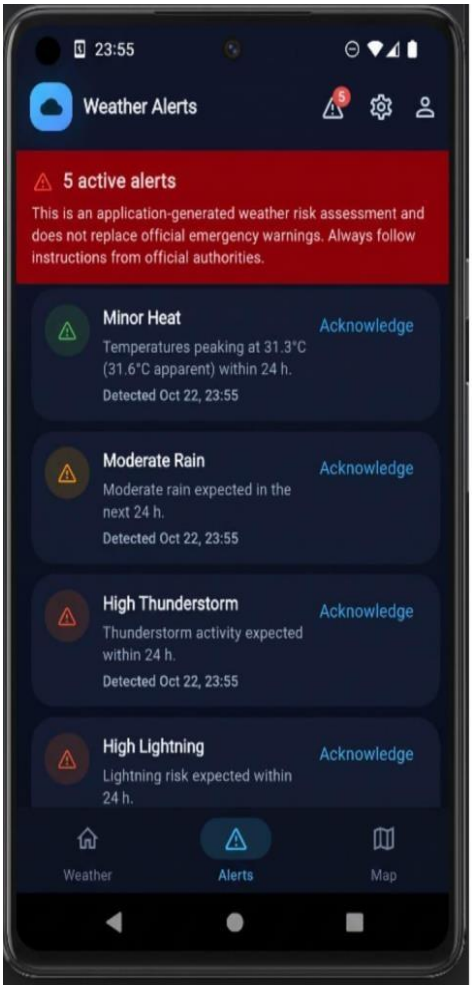
Register Screen



Home Screen



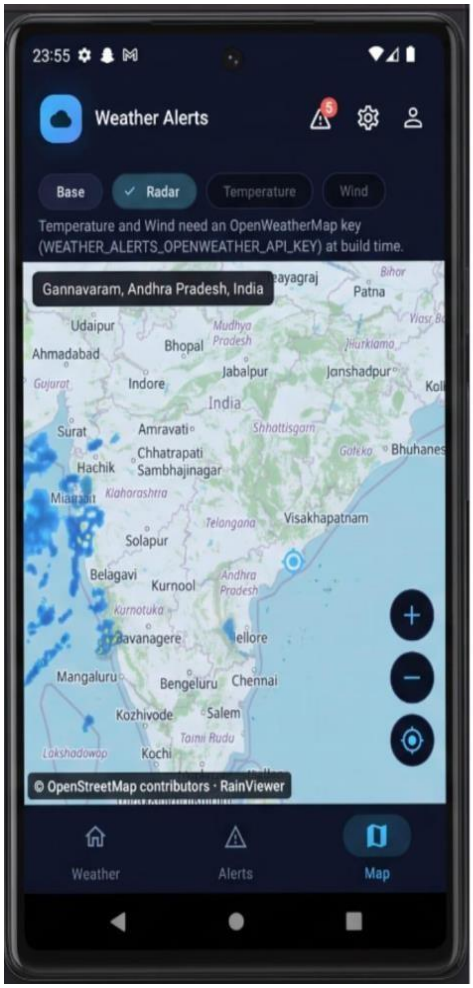
Daily Weather Screen



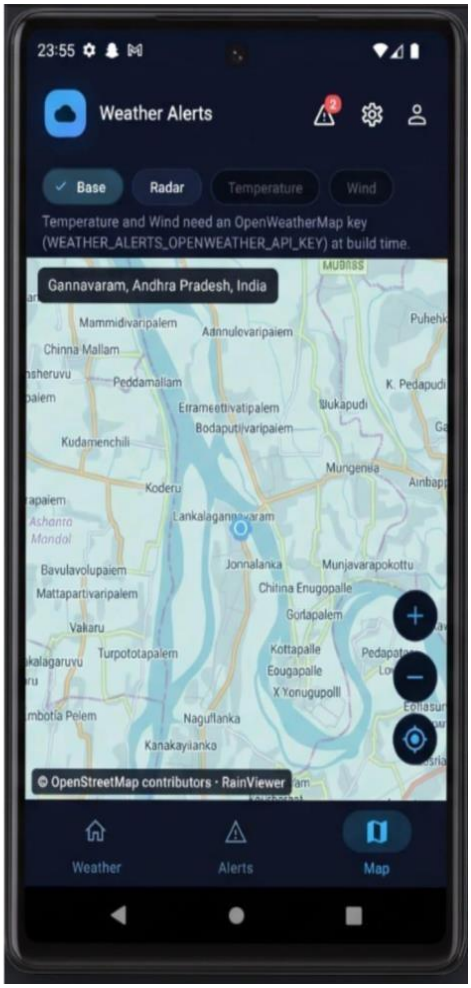
Alerts Screen



Weather Screen



Radar Map Screen



Normal Map Screen

20 Conclusion

Through these experiments, we explored core Flutter principles such as widgets, layouts, navigation, animations, and state management. The Quiz Leaderboard App effectively demonstrates the integration of these concepts in a complete functional project. It showcases how Flutter's declarative UI and reactive framework can be used to build dynamic, responsive, and visually appealing applications.

Additionally, the project provided hands-on experience with API integration, user authentication, and real-time data updates, enhancing our understanding of full-stack mobile app development. The use of Material Design components and custom themes ensured a consistent and engaging user interface. Overall, this project not only strengthened our technical skills in Flutter and Dart but also improved our ability to design and develop scalable mobile applications efficiently.

This project brought together the main stages of Flutter application development: environment setup, language fundamentals, widget composition, responsive layout, navigation, state management, forms, animations, API integration, and testing. Each stage contributed to a more complete understanding of how a modern cross-platform application is designed and maintained.

The Weather Alerts application demonstrates how theoretical concepts translate into practical features. Reusable widgets support consistency, responsive layouts accommodate different devices, navigation connects screens into a coherent flow, and asynchronous data handling keeps the interface responsive while weather information is retrieved.

The project also revealed areas for future improvement. Push notifications could deliver alerts when the application is not open, offline caching could improve resilience, localisation could support additional languages, and automated testing could be expanded to cover more workflows. These enhancements would build on the existing structure without changing its core design.

Overall, the project provided practical experience with Flutter and Dart while reinforcing the importance of clean architecture, careful state management, and user-centred interface design. The resulting application is a solid foundation for further development and a useful demonstration of cross-platform mobile development.

The main limitations of the project are typical of an academic prototype: the data source may be rate-limited, the alert rules are simplified, and testing focuses on representative workflows rather than exhaustive device coverage. These constraints are documented so that the results are interpreted correctly.

Future work could add background notifications, offline storage, localisation, accessibility audits, and automated integration tests. Each improvement would build on the existing architecture while keeping the current screens and data model stable.